

MPI

HIGH PERFORMANCE AND CLOUD COMPUTING

28 April 2026

Indice

0.1	Lecture 1 — Introduction to High Performance Computing (HPC)	3
0.1.1	1. Prologue: Why and Where HPC?	3
0.1.2	2. What is HPC?	6
0.1.3	3. Performance and Metrics	7
0.1.4	4. Supercomputers and the TOP500	9
0.1.5	5. Parallel Computers — Preview	11
0.1.6	References and Resources	11
0.2	Lecture 2 — HPC Hardware and Software	11
0.2.1	6. The Fundamental Equation: $HPC = \text{Parallel Computing}$	11
0.2.2	7. Serial Computers and the Von Neumann Architecture	12
0.2.3	8. Moore’s Law and Dennard Scaling	13
0.2.4	9. Parallel Computers	15
0.2.5	10. HPC Cluster Architecture	17
0.2.6	11. Memory Models in HPC	19
0.2.7	12. Parallelism Within an HPC Node	20
0.2.8	13. HPC Network Infrastructure	21
0.2.9	14. Case Study: The ORFEO HPC Cluster	21
0.2.10	Summary of Key Concepts — Lecture 2	23
0.3	Lecture 3 — ORFEO Infrastructure Updates and the HPC Software Stack	24
0.3.1	15. ORFEO Cluster: Post-PNRR Expansion	24
0.3.2	16. The HPC Software Stack	27
0.3.3	17. Resource Management: The Batch Scheduling System	29
0.3.4	18. Scientific Software Management	33

0.3.5	19. Compilers in HPC	34
0.3.6	20. Libraries: Static vs. Dynamic Linking	35
0.3.7	Summary of Key Concepts — Lecture 3	37
0.4	Lecture 4 — Parallel Concepts and Performance Evaluation	38
0.4.1	21. Parallel Programming Paradigms	38
0.4.2	22. Fundamental Concepts in Parallel Computing	40
0.4.3	23. Parallel Performance Metrics	42
0.4.4	24. Amdahl’s Law	44
0.4.5	25. Gustafson’s Law	46
0.4.6	26. Sustained Performance in Practice	47
0.4.7	27. Sources of Performance Degradation	48
0.4.8	Summary of Key Concepts — Lecture 4	49
0.5	Lecture 5 — Message Passing Programming with MPI	50
0.5.1	28. The Message Passing Interface (MPI)	50
0.5.2	29. Programming with MPI: Fundamentals	52
0.5.3	30. MPI Initialisation and Environment	53
0.5.4	31. MPI Communicators	55
0.5.5	32. Point-to-Point Communication	57
0.5.6	33. Blocking vs. Non-Blocking Communication	60
0.5.7	34. Detailed Send Modes	61
0.5.8	35. Non-Blocking Communication in Detail	63
0.5.9	36. Deadlock in MPI	64
0.5.10	Summary of Key Concepts — Lecture 5	67
0.6	Lecture 6 — MPI Collective Operations	68
0.6.1	37. Introduction to Collective Operations	68
0.6.2	38. Synchronisation: MPI_Barrier	69
0.6.3	39. Data Movement Operations	70
0.6.4	40. Global Computation Operations	76
0.6.5	41. Performance Considerations for Collective Operations	78
0.6.6	Summary of Key Concepts — Lecture 6	80
0.7	Lecture 7 — Network Fundamentals for Parallel Computing	81
0.7.1	42. Modelling Network Performance	81
0.7.2	43. Network Topology	83
0.7.3	44. High-Speed Interconnects in Modern HPC	86

0.7.4	45. ORFEO Network: Detailed Configuration	88
0.7.5	46. Implications for MPI Application Performance	90
0.7.6	Summary of Key Concepts — Lecture 7	91

0.1 Lecture 1 — Introduction to High Performance Computing (HPC)

0.1.1 1. Prologue: Why and Where HPC?

0.1.1.1 1.1 The Scale of Modern Data The contemporary digital landscape is characterised by an unprecedented rate of data generation. In 2026, global daily data production is estimated at approximately **402 to 657 million terabytes** (0.4–0.66 zettabytes), contributing to an annual output projected at 230–240 zettabytes. To put this in perspective, a typical consumer laptop features a hard drive of 256 GB–1 TB and 8–32 GB of RAM — figures that are dwarfed by the volumes of data that modern science and industry must process.

These numbers motivate a fundamental question: **how can we efficiently store, move, and compute over datasets that vastly exceed the capacity of any single machine?** The answer, in large part, is High Performance Computing.

0.1.1.2 1.2 Traditional and Modern Roles of HPC Historically, **supercomputers** were confined to research laboratories and academic institutions, serving disciplines such as theoretical physics, meteorology, and aerospace engineering. Today, HPC has become a pervasive **enabler** across both science and business:

- **Scientific simulation:** weather forecasting, climate modelling, molecular dynamics, astrophysics.
- **Engineering design:** computational fluid dynamics (CFD), structural analysis, crash simulations.
- **Data-intensive analytics:** genomics, financial modelling, large-scale machine learning.
- **Artificial Intelligence:** training large language models (LLMs), protein structure prediction, autonomous systems.

A key insight is that modern HPC **does not necessarily mean supercomputers**. Cloud-based HPC, GPU clusters, and even distributed volunteer computing all fall under the HPC umbrella.

0.1.1.3 1.3 Motivating Examples

0.1.1.3.1 Example 1 — Weather Forecasting and Climate Modelling Weather prediction follows a well-established computational recipe:

1. Define a **mathematical model** (e.g., Navier–Stokes equations for fluid dynamics coupled with thermodynamic equations).
2. **Discretise** the domain over a three-dimensional grid.
3. **Integrate** the governing equations forward in time.
4. **Validate** results against observations.

The computational cost is directly tied to **spatial resolution**. For climate change simulations over the Mediterranean Sea:

Resolution	Approximate RAM Requirement
200 km	~1 GB
25 km	Significantly more (scales roughly as the cube of the resolution ratio)
2 km	Orders of magnitude larger

Reducing the grid spacing from 200 km to 2 km increases the number of grid points by a factor of $(200/2)^3 = 10^6$ in three dimensions, leading to a dramatic increase in both memory and compute requirements. This illustrates why **simulation** has become the third pillar of science, alongside theory and experiment.

0.1.1.3.2 Example 2 — Protein Folding The **protein folding problem** — determining a protein’s three-dimensional structure from its amino acid sequence — exemplifies two complementary HPC paradigms:

- **Approach A — Molecular Dynamics (MD) simulation** (*Folding@home*): a distributed computing project running since the early 2000s, aggregating volunteer CPU and GPU cycles worldwide. As of recent statistics, the project sustains over 128 PFlops (CPU) and 73 PFlops (GPU) of combined throughput. This brute-force approach simulates the physical folding process.
- **Approach B — AI-based prediction** (*AlphaFold*): developed by DeepMind, AlphaFold uses deep learning to predict protein structures directly. It won the CASP (Critical Assessment of protein Structure Prediction) competition in both 2018 and 2020, achieving accuracy comparable to experimental methods. The final release, **AlphaFold 3**, was published in 2024. The AlphaFold Database (July 2022) provides open access to over **200 million** predicted structures (23 TiB of data, released under a CC-BY-4.0 licence).

AlphaFold’s training requirements highlight the scale of modern HPC workloads: the model was trained on **128 TPU v3 cores** (Google’s Tensor Processing Units, application-specific integrated circuits designed for machine learning), with an initial training stage of approximately one week followed by four additional days of fine-tuning.

0.1.1.3.3 Example 3 — Large Language Models (LLMs) Training modern LLMs demands enormous computational resources. A widely used heuristic for estimating training cost, developed by OpenAI and adopted by Google for PaLM 2, is:

$$\text{FLOPs} = 6ND$$

where: - N = number of model parameters, - D = number of training tokens.

Worked example (GPT-3 scale): with $N = 175 \times 10^9$ parameters and $D = 300 \times 10^9$ tokens:

$$\text{FLOPs} = 6 \times 175 \times 10^9 \times 300 \times 10^9 = 3.15 \times 10^{23}$$

This staggering figure — over 10^{23} floating-point operations — underscores why LLM training requires thousands of GPUs running in parallel for weeks or months.

0.1.1.4 1.4 The Convergence of HPC and AI The boundary between traditional HPC and AI workloads is rapidly blurring. Organisations now define HPC broadly to encompass:

- **Traditional HPC:** simulation and modelling (e.g., CFD, finite element analysis).
- **High-Performance Data Analytics (HPDA):** large-scale data processing and analytics.
- **Artificial Intelligence:** deep learning training and inference at scale.

This convergence demands hardware and software ecosystems that can efficiently serve all three workload types, driving the development of **converged HPC/AI clusters**.

0.1.1.5 1.5 The Data Deluge and Data-Intensive Science Data-intensive computing represents a paradigm shift. As data volumes grow from 33 zettabytes (2018) to an expected 181 zettabytes (2025), an increasing number of applications are **data-bound** rather than compute-bound. HPC is essential for processing and analysing this growing volume of data to benefit citizens, businesses, researchers, and public administrations.

The concept of **data-intensive science** (sometimes called the “fourth paradigm” of science, after empirical observation, theoretical modelling, and computational simulation) treats data exploration and analysis as a primary mode of scientific discovery.

0.1.1.6 1.6 The Inference Spiral of System Science There exists a positive feedback loop — the **inference spiral** — in computational science: as models grow more complex and new data sources bring richer information, the demand for computational power increases, which in turn enables more sophisticated models and the ingestion of more data. This spiral is a fundamental driver of HPC evolution.

0.1.2 2. What is HPC?

0.1.2.1 2.1 Definitions Providing a single, precise definition of HPC is challenging. Two complementary definitions capture its scope:

Definition 1 (EU Digital Single Market): High Performance Computing (HPC), also known as supercomputing, refers to computing systems with extremely high computational power that are able to solve hugely complex and demanding problems.

Definition 2 (Industry-oriented): High Performance Computing (HPC) is the use of servers, clusters, and supercomputers — plus associated software, tools, components, storage, and services — for scientific, engineering, or analytical tasks that are particularly intensive in computation, memory usage, or data management. HPC is used by scientists and engineers both in research and in production across industry, government, and academia.

0.1.2.2 2.2 The HPC Ecosystem The second definition highlights three pillars of the HPC ecosystem:

Pillar	Components
Hardware	Servers, clusters, supercomputers, computational accelerators (GPUs, TPUs, FPGAs), high-speed networks (InfiniBand, Ethernet), high-end parallel storage
Software	Middleware (job schedulers, resource managers), scientific/technical libraries, data analysis frameworks, parallel programming tools
Problems	Scientific research, engineering simulation, data analytics, AI/ML training

A more detailed inventory of HPC items includes:

- **Computational accelerators:** GPUs (NVIDIA, AMD), TPUs (Google), FPGAs — devices that offload specific workloads from general-purpose CPUs.
- **High-speed networks:** InfiniBand (NDR, HDR), high-speed Ethernet — enabling low-latency, high-bandwidth communication between compute nodes.
- **High-end parallel storage:** parallel file systems (Lustre, GPFS/Spectrum Scale) designed for concurrent access from thousands of nodes.
- **Middleware:** job schedulers (SLURM, PBS), compilers, MPI libraries, performance analysis tools.

0.1.2.3 2.3 The Human Factor **Human capital** is arguably the most critical component of the HPC ecosystem. Two principal roles exist:

- **HPC providers** (system administrators, infrastructure engineers): plan, install, configure, and manage HPC resources.
- **HPC users** (scientists, engineers, data scientists): develop and run applications that exploit HPC capabilities.

An important observation is that **the interplay between these roles increases overall competence levels**. Users who understand the underlying hardware make better software design decisions; providers who understand application requirements build more effective systems.

0.1.2.4 2.4 HPC as a Tripartite Discipline A holistic view frames HPC as the intersection of three disciplines:

1. **Technology**: hardware architectures, networking, storage systems.
2. **Methodology**: algorithms, programming models, performance optimisation techniques.
3. **Application**: domain-specific scientific and engineering problems.

The **main defining property** and value provided by HPC is **delivering performance for end-user applications** — not performance in the abstract, but performance that translates into faster, larger, or more accurate solutions to real problems.

0.1.3 3. Performance and Metrics

0.1.3.1 3.1 The Centrality of Performance Performance is at the core of HPC as a discipline, yet it is surprisingly difficult to define precisely. Terms like “speed” or “how fast” are inherently vague without specifying:

- **What** is being measured (CPU throughput, memory bandwidth, I/O rate, time-to-solution).
- **How** the measurement is conducted (benchmark, real application, theoretical analysis).
- **For whom** the measurement is meaningful (hardware designer, application developer, end user).

0.1.3.2 3.2 Performance vs. Productivity vs. Profitability The “P” in HPC can stand for more than just Performance:

- **Performance**: raw computational throughput or time-to-solution.
- **Productivity**: a composite metric defined as:

$$\text{Productivity} = \frac{\text{Application Performance}}{\text{Application Programming Effort}}$$

For example, if achieving a $2\times$ speedup requires six months of developer effort, the **productivity gain may not justify the investment**. Different stakeholders (researchers, software engineers, managers) have different productivity expectations.

- **Profitability:** in commercial settings, the return on investment in HPC infrastructure.

Understanding which form of “P” is most relevant to a given context is essential for making sound HPC decisions.

0.1.3.3 3.3 Measuring Basic HPC Performance Three fundamental performance dimensions characterise an HPC system:

1. **Compute performance:** how fast can the system perform arithmetic operations?
2. **Data movement performance:** how fast can data be transferred?
 - CPU Memory (memory bandwidth)
 - CPU Disk (I/O bandwidth)
 - CPU CPU across nodes (network bandwidth and latency)
3. **Storage capacity:** how much data can the system hold?

0.1.3.4 3.4 FLOPS: Floating-Point Operations Per Second The standard metric for computational throughput is **FLOPS** (Floating-Point Operations Per Second), commonly expressed as:

Unit	Value
MFLOPS (MegaFLOPS)	10^6 FLOPS
GFLOPS (GigaFLOPS)	10^9 FLOPS
TFLOPS (TeraFLOPS)	10^{12} FLOPS
PFLOPS (PetaFLOPS)	10^{15} FLOPS
EFLOPS (ExaFLOPS)	10^{18} FLOPS

0.1.3.4.1 Theoretical Peak Performance The **theoretical peak performance** is an upper bound determined by hardware specifications:

$$\text{FLOPS}_{\text{peak}} = f_{\text{clock}} \times N_{\text{FP_ops/cycle}} \times N_{\text{cores}}$$

where: - f_{clock} = clock frequency (cycles per second), - $N_{\text{FP_ops/cycle}}$ = number of floating-point operations completable per cycle (depends on the instruction set, e.g., AVX-512 can perform multiple FP operations per cycle), - N_{cores} = total number of cores.

0.1.3.4.2 Sustained (Real) Performance The **sustained performance** reflects actual application behaviour:

$$\text{FLOPS}_{\text{sustained}} = \frac{\text{Total FP operations in a program}}{\text{Wall-clock execution time (seconds)}}$$

In practice, sustained performance is always **lower** than theoretical peak because of memory access latency, cache misses, branch mispredictions, communication overhead, and load imbalance. The ratio of sustained to peak performance is a measure of **computational efficiency**.

Note: Counting the exact number of floating-point operations in a real application is non-trivial. This is why standardised **benchmarks** are used for fair comparison.

0.1.4 4. Supercomputers and the TOP500

0.1.4.1 4.1 The TOP500 List The **TOP500** list (www.top500.org) is the most widely recognised ranking of the world's most powerful supercomputers. Key facts:

- Published **twice per year** since 1993:
 - **June:** at the ISC (International Supercomputing Conference) in Europe.
 - **November:** at the SC (Supercomputing) conference in the United States.
- Ranking criterion: performance on the **HPL (High-Performance Linpack)** benchmark.

0.1.4.2 4.2 The HPL Benchmark The **HPL** (High-Performance Linpack) benchmark solves a dense system of linear equations using LU factorisation with partial pivoting. Key characteristics:

- Solves $Ax = b$ where A is a uniformly random $n \times n$ matrix.
- The operation count is:

$$\text{FP operations} = \frac{2}{3}n^3 + 2n^2$$

- Results report both the **execution time** and the **floating-point execution rate**.

For each machine on the TOP500, three key metrics are reported:

Metric	Description
$R_{\max}$	Achieved performance (GFLOPS) for the largest problem run
R_{peak}	Theoretical peak performance (GFLOPS)
Power	Electrical power required to run the benchmark (kW)

0.1.4.3 4.3 The 66th TOP500 List — Highlights The 66th edition of the TOP500 list marked several notable milestones:

JUPITER — First European Exascale System: - Installed at the EuroHPC/Jülich Supercomputing Centre (Germany). - Achieved **1.0 ExaFLOP/s** (10^{18} FLOPS), becoming the **fourth exascale system worldwide** and the **first outside the United States**. - Represents Europe's flagship system for scientific and industrial innovation.

Other Notable Systems:

Rank	System	Location	Performance
#5	Microsoft Eagle	Cloud-based	561 PFLOPS
#6	HPC6	Italy	478 PFLOPS
#7	Fugaku	Japan	Remains leading Asian system
#8	Alps	Switzerland	435 PFLOPS
#9	LUMI	Finland	380 PFLOPS
#10	Leonardo	Italy	241 PFLOPS

Microsoft Eagle’s presence at #5 demonstrates the growing viability of **cloud-based HPC** at the highest performance tiers.

0.1.4.4 4.4 The Green500 While the TOP500 ranks systems by raw speed, the **Green500** ranks them by **energy efficiency**, measured in GFLOPS per watt (GFLOPS/W).

Historical Timeline:

Year	Milestone
2005	Concept proposed by Wu-chun Feng (HPPAC keynote)
2007	First official Green500 list released at SC07
2009	Introduction of sub-lists (Little Green500, Open Green500)
2010–2013	Standardised power-measurement methodology adopted
2016	Green500 officially merged with TOP500

Over a 15-year period, the energy efficiency of the greenest supercomputer improved by a factor of **175×**, reflecting the enormous progress in power-efficient computing.

Current Green500 Leaders (66th list):

Rank	System	Location	Efficiency	Performance
#1	KAIROS	CALMIP/University of Toulouse (France)	73.28 GFLOPS/W	3.05 PFLOPS
#2	ROMEO-2025	ROMEO HPC Center (France)	70.90 GFLOPS/W	9.86 PFLOPS
#3	Levante GPU Extension	DKRZ (Germany)	69.43 GFLOPS/W	6.75 PFLOPS

All three systems share the **BullSequana XH3000** platform with **NVIDIA Grace Hopper Superchips** and **NVIDIA NDR200 InfiniBand** interconnects, highlighting the current state-of-the-art balance between energy efficiency and computational performance.

0.1.4.5 4.5 Operating Systems and Performance Trends The TOP500 list reveals a striking dominance of **Linux** as the operating system of choice for supercomputers — virtually all systems in the current list run some variant of Linux, owing to its flexibility, open-source nature, and strong support for parallel computing frameworks.

The historical **performance development** curve of the TOP500 shows consistent exponential growth in both peak and aggregate performance, roughly following a trajectory that doubles the performance of the top system every 1–2 years.

0.1.5 5. Parallel Computers — Preview

The final agenda item — **parallel computers** — sets the stage for subsequent lectures. The core insight motivating parallel computing is that single-processor performance improvement has slowed dramatically (due to power and thermal constraints, often referred to as the end of **Dennard scaling**). To continue increasing computational capability, modern HPC systems rely on **massive parallelism**: distributing work across thousands or millions of processing elements operating simultaneously.

Subsequent lectures will cover parallel architectures (multicore CPUs, GPU accelerators, memory hierarchies, high-speed interconnects) and parallel programming paradigms (MPI, CUDA).

0.1.6 References and Resources

- TOP500 List: www.top500.org
 - HPL Benchmark: <http://icl.cs.utk.edu/hpl/>
 - Green500 retrospective: *Looking Back to Look Forward: 15 Years of the Green500*, IEEE Computer, 2025.
 - AlphaFold Database: <https://alphafold.ebi.ac.uk>
 - Awesome HPC resources: <https://github.com/trevor-vincent/awesome-high-performance-computing>
 - EU HPC portal: <https://ec.europa.eu/digital-single-market/en/high-performance-computing>
 - FLOPS estimation heuristic: Stevens, I. W., *Regulating AI: The Limits of FLOPs as a Metric*, Medium.
-

0.2 Lecture 2 — HPC Hardware and Software

0.2.1 6. The Fundamental Equation: HPC = Parallel Computing

The single most important conceptual identity in High Performance Computing is:

$$\text{HPC} = \text{Parallel Computing}$$

This equivalence is not merely a slogan — it reflects a deep architectural reality. Every modern HPC system, from a multicore laptop to an exascale supercomputer, derives its computational power from **parallelism**: the simultaneous execution of multiple operations, instructions, or tasks. Understanding why this is the case requires tracing the evolution from serial computers through the physical limits that forced the transition to parallel architectures.

0.2.2 7. Serial Computers and the Von Neumann Architecture

0.2.2.1 7.1 The Von Neumann Model The **Von Neumann architecture**, proposed by John von Neumann in 1945, is the foundational model for virtually all general-purpose computers. Its defining characteristics are:

- **Single processing unit (CPU)** comprising:
 - **Control Unit (CU)**: fetches and decodes instructions from memory, orchestrating the execution sequence.
 - **Arithmetic Logic Unit (ALU)**: performs mathematical (addition, multiplication, etc.) and logical (AND, OR, comparison) operations.
 - **Registers**: small, fast storage locations within the CPU (e.g., Program Counter (PC), Instruction Pointer (IP), Stack Pointer (SP)) used to hold the current instruction, operands, and intermediate results.
- **Flat memory model**: memory is treated as a uniform, linear address space. In the idealized model, accessing any memory location has the same cost, and memory access latency is comparable to the time required to execute an arithmetic operation.
- **Stored-program concept**: both instructions and data reside in the same memory, allowing programs to be modified dynamically.

0.2.2.2 7.2 The Instruction Execution Cycle Every instruction in the Von Neumann architecture follows a **five-step workflow**:

1. **Instruction Fetch (IF)**: retrieve the next instruction from memory at the address indicated by the Program Counter.
2. **Instruction Decode (ID)**: interpret the instruction to determine the operation to perform and identify the operands.
3. **Memory Fetch (MF)**: retrieve the required operands from memory (or registers).
4. **Execute (EX)**: perform the specified operation in the ALU.
5. **Write Back (WB)**: store the result back to a register or memory location.

After step 5, the cycle repeats with the next instruction. In a pure serial machine, **only one instruction is in flight at any given time**, creating a fundamental throughput bottleneck.

0.2.2.3 7.3 Instruction Set Architecture (ISA) The **Instruction Set Architecture (ISA)** is the abstract interface between software and hardware — the deepest level of the system that is directly accessible to programmers (or, more commonly, to compilers). Key properties include:

- The ISA defines the set of instructions, data types, registers, addressing modes, and memory architecture that a processor supports.
- It serves as a **contract**: any microarchitecture that implements a given ISA will correctly execute programs written for that ISA.
- **Different microarchitectures can share the same ISA** (they are **binary compatible**). For example, Intel’s Skylake and Alder Lake microarchitectures both implement the x86-64 ISA.
- Successive generations of microarchitectures are typically **backward compatible**, meaning newer processors can run older binaries.
- In the HPC context, the dominant ISA family is **x86** (specifically x86-64), although ARM-based ISAs are increasingly present (e.g., in Fugaku’s A64FX processors and NVIDIA Grace CPUs).

0.2.2.4 7.4 From High-Level Code to Machine Instructions To illustrate the relationship between source code and ISA-level execution, consider a minimal C function:

```
void store(double *a, double *b, double *c) {  
    *c = *a + *b;  
}
```

Compiling with `gcc -O2 -S` on an x86-64 platform produces the following assembly:

```
store:  
    movsd (%rdi), %xmm0      # Load *a into SSE register xmm0  
    addsd (%rsi), %xmm0      # Load *b and add to xmm0  
    movsd %xmm0, (%rdx)      # Store result to *c  
    ret
```

This example reveals that even a single floating-point addition involves **three memory operations** (two loads and one store) plus one arithmetic instruction. The ratio of memory access to computation is a recurring theme in HPC performance analysis; it foreshadows the **memory wall problem** discussed below.

0.2.3 8. Moore’s Law and Dennard Scaling

0.2.3.1 8.1 Moore’s Law **Moore’s Law** is an empirical observation made by Gordon Moore (co-founder of Intel) in 1965:

“The complexity for minimum component costs has increased at a rate of roughly a factor of two per year. [...] Over the longer term, the rate of increase is a bit more uncertain, although there is no reason to believe it will not remain nearly constant for at least 10 years.” — Gordon Moore, *Electronics*, 1965

The law is often popularly (and imprecisely) stated as “performance doubles every 18–24 months.” More accurately, it states that **the number of transistors per unit cost doubles approximately every two years**. The connection to performance is indirect and depends on an additional physical observation: Dennard scaling.

0.2.3.2 8.2 Dennard Scaling Dennard scaling (Robert H. Dennard, 1974) provides the bridge between transistor density and performance:

“Power density stays constant as transistors get smaller.”

The intuition is as follows:

- **Smaller transistors** → shorter signal propagation delays → higher achievable clock **frequency**.
- **Smaller transistors** → lower gate capacitance → lower operating **voltage**.

Since dynamic power consumption in CMOS circuits follows:

$$P \propto C \times V^2 \times f$$

where: - C = capacitance (proportional to transistor size), - V = supply voltage, - f = clock frequency,

shrinking transistors reduces both C and V , which compensates for the increase in f . The net effect is that **power density remains approximately constant** even as clock speed increases. Combined with Moore’s Law, this yielded a golden era (roughly 1970–2005) during which:

Moore’s Law + Dennard Scaling $\Rightarrow$ Faster performance at constant power

0.2.3.3 8.3 The End of Dennard Scaling Around 2004–2006, Dennard scaling **broke down**. At nanometre-scale feature sizes, **leakage currents** (quantum tunnelling through ultra-thin gate oxides) became significant, preventing further voltage reduction. The consequences were stark:

- Increasing clock frequency now **directly increases power consumption and heat dissipation**.
- Thermal limits (the “power wall”) make it impractical to push frequencies much beyond 4–5 GHz with conventional cooling.

This left the industry with two options:

1. **Increase power** to sustain frequency scaling — impractical due to thermal and energy constraints.
2. **Stop frequency scaling** and find alternative paths to performance — the option that was adopted.

0.2.3.4 8.4 The Multicore Revolution While Dennard scaling ended, **Moore’s Law continued**: transistor counts kept doubling. The question became: *what to do with all those transistors if not to increase clock speed?*

The answer was **multicore processors**: instead of making a single core faster, manufacturers began placing **multiple independent cores** on a single chip. Key characteristics of the multicore approach:

- Clock frequencies have remained roughly constant (or even decreased slightly) since ~2005.
- Core counts have steadily increased: from dual-core (2005) to 64+ cores per socket in modern server processors (e.g., AMD EPYC 7H12 with 64 cores).
- Cores on the same chip **share certain resources** (last-level cache, memory controllers, network interfaces, disk I/O) but are capable of **independent computation**.
- Performance gains now depend on **effective exploitation of thread-level parallelism** — software must be explicitly designed to use multiple cores.

This architectural shift is the fundamental reason why **HPC is synonymous with parallel computing**: sequential performance improvement has plateaued, and all further gains come from parallelism.

0.2.4 9. Parallel Computers

0.2.4.1 9.1 Flynn’s Taxonomy **Flynn’s taxonomy** (Michael J. Flynn, 1966) classifies computer architectures along two dimensions:

- **Instruction stream**: the sequence of instructions executed by the processor.
- **Data stream**: the sequence of data items operated upon.

This yields four categories:

Category	Instruction Streams	Data Streams	Description
SISD	Single	Single	Classical Von Neumann machine; one instruction operates on one data item at a time.
MISD	Multiple	Single	Multiple instructions operate on the same data stream. Rare in practice; sometimes cited for fault-tolerant systems or pipeline architectures.

Category	Instruction Streams	Data Streams	Description
SIMD	Single	Multiple	One instruction operates on multiple data items simultaneously. Examples: vector units within CPU cores (SSE, AVX-512), GPUs, vector processors, FPGAs.
MIMD	Multiple	Multiple	Multiple independent processors execute different instructions on different data. This is the dominant model for modern multicore and multi-node HPC systems.

0.2.4.2 9.2 Mapping Flynn’s Taxonomy to Modern Hardware and Software Although Flynn’s taxonomy was formulated over 50 years ago and does not capture the full complexity of modern HPC architectures, the **SIMD** and **MIMD** concepts remain fundamental:

Category	Hardware Level	Software Level
SISD	A single Von Neumann CPU (no parallelism)	No parallelism at all
MISD	On a superscalar CPU, different execution ports processing the same data	Instruction-Level Parallelism (ILP) on the same data; multiple tasks/threads operating on the same data
SIMD	Any vector-capable hardware: vector registers on a core, a GPU, a vector processor, an FPGA	Data parallelism through vector instructions and operations
MIMD	Every multicore/multiprocessor system; on a superscalar CPU, different ports executing different ops on different data	ILP on different data; multiple tasks or threads executing different code on different data

Modern HPC systems are fundamentally **MIMD** machines at the node and cluster level, with **SIMD** parallelism exploited at the instruction level within each core.

0.2.5 10. HPC Cluster Architecture

0.2.5.1 10.1 Essential Components of an HPC Cluster An HPC cluster is a collection of interconnected computers (called **nodes**) that work together to solve large computational problems. The essential components are:

1. **Compute nodes:** individual servers, often mounted in standard **1U rack-mount** form factor for dense packing.
2. **Interconnection network(s):** one or more high-speed networks linking the nodes together.
3. **Storage subsystem:** shared parallel file systems accessible from all nodes.
4. **Login/access node(s):** gateway nodes where users log in, compile code, and submit jobs.

The hierarchical structure of an HPC system spans multiple levels:

Core → CPU (Socket) → Node → Rack → Cluster

0.2.5.2 10.2 The CPU: Multicore Topology A modern CPU is a **multicore processor** — a single chip containing multiple independent processing cores. Each core is defined as:

Core: the smallest unit of computing, having one or more hardware/software threads and responsible for executing instructions.

0.2.5.2.1 Cache Hierarchy Modern CPUs employ a **multi-level cache hierarchy** to mitigate the latency gap between fast cores and slow main memory. The typical hierarchy is:

Cache Level	Scope	Typical Size	Typical Latency
L1d / L1i (data / instruction)	Private to each core	32–64 KB each	~1–4 cycles
L2	Private to each core (or shared by a pair of cores)	256 KB – 1 MB	~10–15 cycles
L3 (Last-Level Cache, LLC)	Shared across all cores in a socket	8–256 MB	~30–50 cycles

Cache topologies vary across architectures. Two common arrangements:

- **Topology A:** Each core has private L1 and L2 caches; pairs of cores share an L2; all cores share a single L3.
- **Topology B:** Each core has private L1 and L2 caches; all cores share a unified L3.

The cache hierarchy has profound implications for performance: data that resides in L1 can be accessed orders of magnitude faster than data in main memory (DRAM).

0.2.5.2.2 CPU Internal Layout: From Front Side Bus to Integrated Memory Controller Older CPU architectures used a **Front Side Bus (FSB)** topology:

- Each core had a private cache.
- All cores communicated with a shared cache via the FSB.
- A **NorthBridge** chip handled communication between the CPU, DRAM, and PCIe devices.
- A **SouthBridge** chip managed slower I/O (USB, SATA, etc.).

Modern CPU architectures have **integrated the NorthBridge** (memory controller) directly onto the CPU die:

- Multiple memory channels (typically 2–8 lanes) connect the CPU directly to DRAM banks, reducing memory access latency.
- The **SouthBridge** (now commonly called the **chipset** or **Platform Controller Hub**) remains as a separate chip handling lower-bandwidth I/O (USB, PCIe for peripherals).

This integration is a critical performance optimisation, as it eliminates the FSB bottleneck and provides each CPU socket with direct, low-latency access to its local DRAM.

0.2.5.3 10.3 Node Topology: Multi-Socket Systems A compute node typically contains **multiple CPU sockets** (commonly 2 or 4). In a multi-socket node:

- Each socket has its own set of cores, private caches, LLC, and local DRAM.
- Sockets are connected via a **cache-coherent interconnect** (e.g., Intel UPI — Ultra Path Interconnect, or AMD Infinity Fabric) that allows any core to access any memory location across all sockets.

0.2.5.4 10.4 Inside a Compute Node A physical 1U compute node typically contains:

Component	Description
CPU sockets (typically 2)	Each populated with a multicore processor
RAM slots	Multiple DIMM slots per socket (e.g., 8–16 per socket) for local DRAM
Disk bays	For local storage (NVMe SSDs or HDDs)
Network card(s)	High-speed interconnect (InfiniBand HCA) and/or Ethernet NICs
Power supply	Redundant PSUs for reliability

0.2.6 11. Memory Models in HPC

0.2.6.1 11.1 The Memory Wall Problem The **memory wall** refers to the growing disparity between processor speed and memory access speed. While CPU clock frequencies and core counts have increased dramatically over decades, DRAM access latency has improved much more slowly. This gap means that processors frequently **stall**, waiting for data to arrive from memory.

The memory wall problem is **aggravated by multicore architectures**:

- **Memory bandwidth** must be shared among all cores on a socket.
- **Cache sharing** at the LLC level introduces contention.
- Getting data out of DRAM banks and into the multicore processor array becomes a critical bottleneck.

Effective HPC programming requires careful attention to **data locality** — ensuring that data accessed by a core resides in nearby (fast) cache levels rather than distant (slow) memory.

0.2.6.2 11.2 Shared Memory: UMA and NUMA Within a single node, memory is **shared** — all cores can access all memory locations using standard load/store instructions. However, there are two fundamentally different shared-memory models:

0.2.6.2.1 UMA — Uniform Memory Access In a **UMA** (also called **Symmetric Multi-processing, SMP**) architecture:

- All processors have **equal access time** to all memory locations.
- A single, unified memory pool serves all cores.
- This model is simple to program but does not scale well beyond a small number of processors due to memory bus contention.

0.2.6.2.2 NUMA — Non-Uniform Memory Access In a **NUMA** architecture:

- Memory is physically distributed across sockets, with each socket having its own **local DRAM**.
- A core accessing its **local memory** experiences lower latency and higher bandwidth than when accessing **remote memory** (i.e., DRAM attached to a different socket).
- Cache coherency protocols ensure correctness across sockets, but remote memory access is significantly slower (typically $1.5\text{--}3\times$ the latency of local access).

Virtually all modern multi-socket servers are NUMA systems. This has important programming implications: for optimal performance, data should be allocated in the memory bank **local to the cores that will process it**. Tools such as `numactl` (Linux utility for controlling NUMA policy), `/proc/cpuinfo`, and `hwloc` (from the OpenMPI project) allow users to discover and manage NUMA topology.

0.2.6.3 11.3 Distributed Memory When computation spans **multiple nodes**, each node’s memory is isolated — a process on one node **cannot directly access** the memory of another node using load/store instructions. Instead, it must issue explicit **communication requests** (typically via message passing) to exchange data. This is called **distributed memory**.

The distinction between shared and distributed memory is both a **hardware concept** (describing physical memory accessibility) and a **programming paradigm**:

Memory Model	Hardware Reality	Programming Paradigm	Typical API
Shared memory	All cores on a node can access all DRAM on that node	Threads share an address space	OpenMP, pthreads
Distributed memory	Each node has private memory; inter-node access requires network communication	Processes communicate via messages	MPI

Most HPC applications use a **hybrid** approach: MPI for inter-node communication (distributed memory) combined with OpenMP or threading for intra-node parallelism (shared memory).

0.2.7 12. Parallelism Within an HPC Node

Modern HPC nodes expose **multiple levels of parallelism**, each exploitable by different hardware and software mechanisms:

Level	Hardware Resource	Description
(1) ILP / SIMD	Instruction-Level Parallelism and vector units (e.g., AVX-512)	Multiple operations per clock cycle within a single core
(2) Cores	Multiple cores per CPU socket	Independent instruction streams executing in parallel
(3) Inner cache levels	Private L1/L2 caches per core	Enable independent data access without contention
(4) Sockets / ccNUMA domains	Multiple CPU sockets per node	Each socket is a NUMA domain with local memory
(5) Accelerators	GPUs (e.g., NVIDIA V100, A100), FPGAs	Massively parallel co-processors for specific workloads

A high-end compute node might contain 2–4 CPU sockets (each with 32–128 cores), 2–8 GPUs, and optionally FPGAs — yielding thousands of parallel execution units within a single node.

0.2.7.1 12.1 A Note on Terminology HPC jargon can be confusing due to overloaded terms:

Term	Meaning
Multiprocessor	A server with more than one CPU (socket)
Multicore	A CPU with more than one core
Processor / CPU / Socket	Often used interchangeably to refer to the physical chip
Processor (alternative usage)	Sometimes used to mean a single core
“A process per processor”	Typically means one process per core

Context is essential for disambiguating these terms in HPC literature and documentation.

0.2.8 13. HPC Network Infrastructure

0.2.8.1 13.1 Network Types in an HPC Cluster An HPC cluster employs multiple networks, each serving a distinct purpose:

Network Type	Purpose	Key Requirements	Typical Technology
High-speed interconnect	Parallel computation (inter-node communication)	Low latency, high bandwidth	InfiniBand (HDR, NDR)
I/O network	Storage access (NFS, parallel file systems)	Good bandwidth; latency less critical	10/25 GbE, InfiniBand
In-band management network	Management services (job scheduler, NFS, software distribution)	Moderate bandwidth	Gigabit Ethernet
Out-of-band management network	Remote hardware control (IPMI/BMC, power cycling, BIOS access)	Reliability	1 GbE (dedicated)

The **high-speed interconnect** is the most performance-critical network. **InfiniBand** is the dominant technology in HPC, offering: - Latency as low as $\sim 1 \mu\text{s}$ (compared to $\sim 10\text{--}50 \mu\text{s}$ for Ethernet). - Bandwidth of 200–400 Gb/s per port (NDR/NDR200). - RDMA (Remote Direct Memory Access) capability, allowing data transfer without CPU involvement.

The overall cluster topology — how nodes are interconnected — significantly affects communication performance. Common topologies include fat-tree, dragonfly, and torus networks.

0.2.9 14. Case Study: The ORFEO HPC Cluster

The **ORFEO** cluster provides a concrete example of a production HPC infrastructure, illustrating how the theoretical concepts discussed above are realised in practice.

0.2.9.1 14.1 Compute Nodes ORFEO comprises a heterogeneous set of compute nodes:

Node Type	Quantity	RAM per Node	Cores per Node	GPUs per Node	Peak Performance (TFLOPS)
THIN (Intel)	10	768 GB	24	—	1.997
FAT (Intel)	2	1,536 GB	36	—	3.456
GPU (Intel)	4	256 GB	24	2× NVIDIA V100 (32 GB)	2.073 + 2×7
EPYC (AMD)	8	512 GB	128 (EPYC 7H12 64-core)	—	—
DGX Station (NVIDIA)	2	2,048 GB	128 (EPYC)	8× NVIDIA A100	—
Totals	~26	~16 TB aggregate	~1,688	24	—

This heterogeneity reflects the diverse workload requirements of a modern HPC centre: **THIN nodes** for general-purpose parallel computing, **FAT nodes** for memory-intensive applications (e.g., large-scale data analytics, genome assembly), **GPU nodes** for accelerated computing (deep learning, molecular dynamics), and **DGX Stations** for the most demanding AI workloads.

0.2.9.2 14.2 Network Configuration ORFEO employs a tiered network architecture:

Network	Technology	Bandwidth
High-speed interconnect	HDR InfiniBand	100 Gbit/s
I/O + In-band management	Ethernet	25 Gbit/s
Out-of-band management	Ethernet	1 Gbit/s

0.2.9.3 14.3 Storage Subsystem ORFEO’s storage is organised into four tiers, reflecting the trade-off between speed, capacity, and cost:

Tier	Technology	Servers	Raw Capacity	File System	Purpose
Fast (NVMe)	NVMe PCIe	4	12 TB	Ceph parallel FS	Ultra-low-latency scratch for I/O-intensive jobs
Fast (SSD)	SSD	6	320 TB	Ceph parallel FS	High-throughput scratch storage

Tier	Technology	Servers	Raw Capacity	File System	Purpose
Standard (HDD)	HDD	1	1,080 TB	Ceph parallel FS	General-purpose storage (home directories, project data)
Long-term preservation	HDD	—	1,512 TB	NFS	Archival storage for processed datasets

All high-performance tiers use **Ceph**, a distributed parallel file system, exported via InfiniBand to all compute nodes for maximum bandwidth. Long-term storage uses **NFS** over Ethernet, prioritising capacity and reliability over performance.

0.2.9.4 14.4 User Storage Areas Users interact with four main storage areas:

Path	Location	Purpose	Quota
/u/<group>/<user> (Home)	Ceph large FS, exported via InfiniBand	User home directory; default landing point after login	2 TB per user
/scratch/<group>/<user>	Ceph large FS, exported via InfiniBand	Temporary storage for data under active processing	Shared pool (~598 TB total)
/fast/<group>/<user>	Ceph fast FS (SSD/NVMe)	High-speed scratch for data-intensive applications	Shared pool (~88 TB total)
/storage/<group>/<user>	NFS via Ethernet	Long-term storage for final processed datasets	Allocated on request

Symbolic links in the home directory provide convenient access to **/fast**, **/storage**, and **/scratch**.

0.2.10 Summary of Key Concepts — Lecture 2

Concept	Key Takeaway
Von Neumann architecture	Single-processor, sequential execution model; foundation of all modern CPUs
ISA	Hardware-software interface; x86-64 dominates HPC

Concept	Key Takeaway
Moore’s Law	Transistor count doubles ~every 2 years; still valid
Dennard scaling	Power density constant with shrinking transistors; ended ~2005
Multicore revolution	Response to end of Dennard scaling; performance through parallelism
Flynn’s taxonomy	SIMD and MIMD remain central to HPC architecture
Cache hierarchy	L1 → L2 → L3; critical for mitigating memory wall
NUMA	Non-uniform memory access in multi-socket nodes; locality matters
Memory models	Shared (intra-node) vs. distributed (inter-node)
HPC cluster	Nodes + interconnect + storage + management

0.3 Lecture 3 — ORFEO Infrastructure Updates and the HPC Software Stack

0.3.1 15. ORFEO Cluster: Post-PNRR Expansion

The ORFEO cluster underwent a significant expansion funded by the Italian **PNRR** (Piano Nazionale di Ripresa e Resilienza). Comparing the pre- and post-expansion configurations reveals the scale of the upgrade.

0.3.1.1 15.1 Compute Nodes: Before and After Pre-PNRR configuration (26 nodes):

Node Type	Qty	RAM / Node	Cores / Node	GPUs / Node	Peak (TFLOPS)
THIN (Intel)	10	768 GB	24	—	1.997
FAT (Intel)	2	1,536 GB	36	—	3.46
GPU (Intel)	4	256 GB	24	2× NVIDIA V100 (32 GB)	2.073 + 2×7.0
EPYC (AMD)	8	512 GB	128 (EPYC 7H12)	—	5.03
DGX Station (NVIDIA)	2	1,024 GB	128 (EPYC)	8× NVIDIA A100 (40 GB)	5.03 + 8×9.7
Totals	26	~15 TB	1,688	24	~290

Note on A100 peak performance: The A100’s peak of 9.7 TFLOPS is reported in **FP64** (double-precision). When using **TF32** (Tensor Float 32), a reduced-precision

format specifically designed for deep learning, the A100 reaches up to **312 TFLOPS** — a dramatic increase that underscores how modern accelerators are optimised for AI workloads.

Post-PNRR configuration (42 nodes):

Node Type	Qty	RAM / Node	Cores / Node	GPUs / Node
THIN (Intel)	10	768 GB	24	—
FAT (Intel)	2	1,536 GB	36	—
GPU (Intel)	4	256 GB	24	2× NVIDIA V100 (32 GB)
EPYC (AMD)	8	512 GB	128	—
DGX Station (NVIDIA)	2	1,024 GB	128	8× NVIDIA A100 (40 GB)
GENOA (AMD)	13	512 GB	64	—
Dell XE9680	3	1,024 GB	112	8× NVIDIA H100 (80 GB)
Totals	42	~25 TB	2,856	48

The expansion delivered substantial improvements across every metric:

Metric	Increase
Compute nodes	+60%
Aggregate RAM	+66%
Total cores	+70%
GPUs	+100% (24 new GPUs, all NVIDIA H100)

The addition of **AMD GENOA** nodes brings the latest Zen 4 architecture to ORFEO, while the **Dell XE9680** nodes equipped with **NVIDIA H100** GPUs (80 GB HBM3 each) represent the current state-of-the-art for AI and HPC accelerated workloads.

0.3.1.2 15.2 Network: Before and After Pre-PNRR:

Network	Technology	Details
HPC interconnect	HDR InfiniBand	100 Gbit/s; 1× QM8700 Mellanox switch (16 Tb/s aggregate throughput, sub-130 ns switch latency)
I/O + In-band management	Ethernet	2× 25 Gbit/s links; 6× Dell S5148F switches (3.6 Tbps aggregate, fully redundant)
Out-of-band management	Ethernet	1 Gbit/s

Post-PNRR:

Network	Technology	Details
HPC interconnect	HDR + HDR200 InfiniBand	100 + 200 Gbit/s; QM8700 + QM9700 Mellanox switches (16 + 51.2 Tb/s aggregate throughput)
I/O + In-band management	Ethernet	2× 25 Gbit/s; 10 × Dell S5148F switches (3.6 Tbps each, fully redundant)
Out-of-band management	Ethernet	1 Gbit/s

The key improvement is the introduction of the **QM9700** switch supporting **200 Gbit/s InfiniBand**, which triples the aggregate switch throughput from 16 to 67.2 Tb/s, essential for scaling communication across the expanded node count.

0.3.1.3 15.3 Storage: Before and After Pre-PNRR:

Tier	Technology	Nodes	Raw Capacity	File System
Standard (HDD)	Gen1 + Gen2 HDDs	6+2	~2,450 TB	Ceph
Fast (SSD)	SSD	4	~340 TB	Ceph
Super Fast (NVMe)	NVMe PCIe	2 devices/node	~32 TB	Ceph
Long-term	HDD	NFS server + SAN	~2,962 TB	NFS

Post-PNRR:

Tier	Technology	Nodes	Raw Capacity	Change
Standard (HDD)	Gen1 + Gen2 + Gen3 HDDs	6+2+ 11	~ 5,684 TB	+ 130%
Fast (SSD)	SSD	4	~340 TB	Unchanged
Super Fast (NVMe)	NVMe PCIe	2 devices/node; Gen3 nodes have 2×14 TB	~ 340 TB	+ 1,340%
Long-term	HDD	NFS server + SAN	~2,962 TB	Unchanged

The most dramatic improvement is in the **NVMe tier**, which expanded by over an order of magnitude — from 32 TB to 340 TB — reflecting the growing importance of ultra-fast I/O for data-intensive workloads.

Additionally, ORFEO's I/O subsystem now includes a **Dell ML3 Tape Library** for archiving data on **LTO-9** cartridges, each offering up to **45 TB** of capacity, providing cost-effective long-term data preservation.

0.3.1.4 15.4 Updated Storage Areas The user-facing storage paths on the updated ORFEO system are:

Path	File System	Technology	Protection	Purpose
/orfeo/cephfs/home	Ceph	SSD	Replica 3	User home directories; 200 GB quota, 10^6 file limit
/orfeo/cephfs/scratch	Ceph (via InfiniBand)	HDD	6+2 Erasure Coding	Temporary storage for data under active processing
/orfeo/cephfs/fast	Ceph	SSD	Replica 3	High-speed scratch for data-intensive applications

Data protection strategies differ by tier: - **Replica 3**: three copies of each data block are stored across different nodes, providing high durability at the cost of $3\times$ raw capacity overhead. Used for home and fast tiers where reliability is paramount. - **6+2 Erasure Coding (EC)**: data is split into 6 data chunks and 2 parity chunks, tolerating the loss of any 2 chunks. This provides good durability with lower overhead ($\sim 33\%$) than replication, suitable for the large scratch tier.

0.3.1.4.1 Benchmarking Storage Performance Users can measure I/O throughput on different storage tiers using the `dd` command:

```
# Write 2 GB with 2 MB block size
dd if=/dev/zero of=testfile bs=2M count=1024 conv=fdatasync

# Write 4 GB with 4 MB block size
dd if=/dev/zero of=testfile bs=4M count=1024 conv=fdatasync
```

The `conv=fdatasync` flag ensures data is physically written to disk before reporting, providing accurate throughput measurements. Running these commands on `/orfeo/cephfs/fast` vs. `/orfeo/cephfs/scratch` reveals the significant performance difference between SSD-backed and HDD-backed Ceph pools.

0.3.2 16. The HPC Software Stack

0.3.2.1 16.1 Layered Architecture An HPC cluster's software is organised into a **layered stack**, where each layer builds upon the services provided by the layers below:

Layer (bottom to top)	Components	Role
Operating System + Services	Linux (CentOS, Rocky, Ubuntu)	Base OS, kernel, device drivers
Network	InfiniBand drivers, IP stack	Fast interconnection among nodes
Storage	Ceph, NFS, parallel FS drivers	Shared and parallel file systems
System Management Software	Installation, administration, monitoring tools	Provisioning, configuration management
Resource Management Software	SLURM, PBS, LSF	Job scheduling, resource allocation
Parallel Environment	MPI libraries (OpenMPI, MVAPICH, MPICH)	Inter-process communication for parallel apps
Compilers	GCC, Intel oneAPI, NVIDIA HPC SDK	Translation from source code to machine code
Mathematical/Scientific Libraries	BLAS, LAPACK, ScaLAPACK, MKL, FFTW	Optimised numerical routines
I/O Libraries	HDF5, NetCDF, parallel I/O	Efficient data input/output
User Applications	Serial and parallel programs	Domain-specific computations

A notable feature of HPC software stacks is the **lack of standardisation**: every machine and application may have a different combination of compilers, libraries, and tools, tuned for maximum performance. This is by design — **HPC frequently trades reuse and usability for performance**.

The consequence is the so-called **dependency nightmare**: installing and maintaining scientific software requires careful tracking of compiler versions, library versions, and their interdependencies.

0.3.2.2 16.2 Cluster-Wide Services Beyond the compute stack, an HPC cluster requires a set of **cluster-wide infrastructure services** that keep the system operational:

Service	Protocol/Tool	Purpose
Time synchronisation	NTP	Ensures all nodes share a consistent clock
Name resolution	DNS	Dynamic hostname resolution across the cluster
Provisioning	DHCP + TFTP	Automated installation and configuration of nodes (including network device configuration and backup)
Shared file system	NFS	Exports shared directories (software, home dirs) to all nodes
Remote access	SSH	Secure login and file transfer
Parallel computation	MPI	Launches and coordinates parallel jobs across nodes

Service	Protocol/Tool	Purpose
Authentication	LDAP / NIS	Centralised user account management
Monitoring	Grafana, Zabbix	Real-time system health and performance monitoring
Hardware management	IPMI, SNMP	Out-of-band remote control of node hardware

These services are typically managed by a **master/server node** that is distinct from the compute nodes.

0.3.2.3 16.3 Cluster Middleware on ORFEO The specific middleware stack deployed on ORFEO illustrates a typical HPC software configuration:

Layer	Components
Languages	Fortran, C/C++, Python
MPI Implementations	MVAPICH, MPICH, OpenMPI
Compilers	Intel (oneAPI), PGI (now NVIDIA HPC SDK), GNU (GCC)
Numerical Libraries	BLAS, LAPACK, ScaLAPACK, ATLAS, MKL, FFTW
Resource Manager	SLURM
Container Platform	OpenStack (cloud layer)
Administration	SSH, C3Tools, ad-hoc utilities, IPMI, SNMP
Monitoring	Grafana, Zabbix
Operating System	Linux (CentOS / Rocky Linux)
Interconnect	InfiniBand (HDR/HDR200)
File Systems	NFS, Ceph

0.3.3 17. Resource Management: The Batch Scheduling System

0.3.3.1 17.1 The Resource Management Problem An HPC cluster serves **multiple users** competing for a **finite pool of resources** (CPU cores, memory, GPUs, network bandwidth, storage I/O). The resource management problem is analogous to fairly dividing a limited resource (like slicing a cake) among competing requests:

1. **Resource tracking:** software that monitors the availability and state of all computational resources.
2. **Scheduling:** software that decides **which job to execute next**, based on available resources, user priorities, and fairness policies.
3. **Execution:** software that actually launches, monitors, and cleans up jobs on the assigned nodes.

The system that performs these functions is called a **Local Resource Management System (LRMS)**, also known as a **batch scheduler** or **workload manager**.

0.3.3.2 17.2 Key Definitions

- **Batch Scheduler:** software responsible for scheduling users' jobs on the cluster. **Scheduling** is the method by which work, specified by some means, is assigned to resources that complete the work.
- **Resource Manager:** software that allocates resources to jobs and manages their execution on compute nodes (launching, monitoring, cleaning up).
- **Compute Node** (also called *computing node*): a server used for its computational power to execute user jobs.
- **Login Node** (also called *master node* or *access node*): the entry point where users log in, compile code, and submit jobs. Users should **never** run computationally intensive tasks on the login node.

0.3.3.3 17.3 LRMS Functional Layers A modern LRMS operates across three functional layers:

Layer	Responsibilities
Resource Management	Launching jobs on nodes, monitoring execution, cleanup after completion
Job Management	Batch and interactive job handling, backfilling, scheduling, suspend/resume, preemption, job dependencies, resubmission, advance reservation
Workload Management	Higher-level scheduling policies: fair-share scheduling (ensuring equitable resource distribution over time), Quality of Service (QoS) classes, Service Level Agreements (SLA) , energy-saving policies

The dual goal of any LRMS is to: - **Satisfy users:** minimise response time (how long a job waits in the queue), maximise reliability. - **Satisfy administrators:** maximise resource utilisation (keep the cluster busy), achieve energy efficiency.

These goals are inherently in tension — a policy that minimises wait time for one user may starve others — making **resource sharing policy a social and organisational problem**, not merely a technical one.

0.3.3.4 17.4 Major LRMS Software Packages

Package	Licence	Notes
IBM LSF (Load Sharing Facility)	Commercial	Widely used in industry
Univa Grid Engine (UGE)	Commercial	Originates from Sun Grid Engine (SGE)
PBS Professional	Open source (formerly commercial)	Commercial support available

Package	Licence	Notes
SLURM	Open source	Commercial support available; dominant in academic HPC

SLURM is the LRMS deployed on ORFEO and is the most widely adopted scheduler in the academic and research HPC world.

0.3.3.5 17.5 SLURM: Simple Linux Utility for Resource Management

0.3.3.5.1 Overview SLURM began in 2002 at **Lawrence Livermore National Laboratory** (LLNL) as a lightweight resource manager for Linux clusters. Sophisticated scheduling plugins were added in 2008, transforming it into a full-featured workload manager. Today, SLURM comprises approximately **550,000 lines of C code** and is used on many of the world’s largest supercomputers.

0.3.3.5.2 Architecture SLURM follows a **centralised controller** architecture:

- **slurmctld** (controller daemon): runs on the management node; maintains the global state of all resources and jobs; makes all scheduling decisions.
- **slurmd** (node daemon): runs on each compute node; receives job assignments from slurmctld, launches and monitors processes, reports status back.
- **slurmdbd** (database daemon, optional): stores accounting and historical job data in a database for reporting and fair-share calculations.

0.3.3.5.3 Core Entities SLURM organises resources and work into five fundamental entities:

Entity	Description
Node	A compute resource (physical or virtual server)
Partition	A logical grouping of nodes (possibly overlapping). Partitions serve a role similar to “queues” in other schedulers — they define access policies, resource limits, and priorities for different classes of jobs
Job	An allocation of resources (nodes, cores, memory, GPUs, time) assigned to a user for a specified duration
Job Step	A set of (possibly parallel) tasks within a job. A single job may contain multiple sequential or concurrent job steps
QoS (Quality of Service)	Defines limits and policies (e.g., maximum wall time, priority boost, resource caps) applied to jobs

0.3.3.5.4 Writing a SLURM Job Script A SLURM job script is a standard **shell script** augmented with special **#SBATCH** directives that specify resource requirements. The scheduler reads these directives to determine how to allocate resources:

```
#!/bin/bash
#SBATCH --ntasks=128          # Total number of MPI tasks
#SBATCH --cpus-per-task=2     # CPU cores per task (for hybrid MPI+OpenMP)
#SBATCH --mem-per-cpu=20      # Memory per CPU core (MB)
#SBATCH --time=60             # Wall-clock time limit (minutes)

cd $HOME/MyJobDir            # Navigate to working directory
hostname                     # Print the node name
pwd                           # Confirm working directory
```

Key points: - SLURM directives use the **#SBATCH** prefix (not **#SLURM**, though some legacy documentation uses the latter). - When a job starts, SLURM places the user in their **home directory** by default — it is essential to explicitly change to the desired working directory. - Common resource specification parameters include:

Parameter	Meaning
<code>--ntasks</code> or <code>-n</code>	Number of parallel tasks (MPI processes)
<code>--cpus-per-task</code> or <code>-c</code>	CPU cores allocated per task
<code>--mem-per-cpu</code>	Memory per CPU core
<code>--mem</code>	Total memory per node
<code>--time</code> or <code>-t</code>	Maximum wall-clock time (formats: minutes, HH:MM:SS, D-HH:MM:SS)
<code>--partition</code> or <code>-p</code>	Target partition
<code>--gres=gpu:N</code>	Request N GPUs
<code>--job-name</code> or <code>-J</code>	Human-readable job name
<code>--output</code> or <code>-o</code>	File for standard output
<code>--error</code> or <code>-e</code>	File for standard error

0.3.3.5.5 Essential SLURM Commands

Command	Purpose
<code>sbatch <script></code>	Submit a batch job script
<code>srun <command></code>	Run a parallel job step (can also be used for interactive jobs)
<code>salloc</code>	Allocate resources for an interactive session
<code>squeue</code>	View the job queue (pending and running jobs)
<code>scancel <jobid></code>	Cancel a job
<code>sinfo</code>	Display partition and node status
<code>sacct</code>	View accounting data for past and current jobs
<code>scontrol show job <id></code>	Detailed information about a specific job

0.3.3.5.6 Practical Observations on LRMS

- **For users:** understanding the LRMS well is essential — it is the gateway to running any computation on the cluster. Properly specifying resource requirements avoids both wasted resources (over-requesting) and job failures (under-requesting).
 - **For administrators:** the LRMS is the primary tool for managing cluster utilisation and enforcing fair-use policies.
 - While modern LRMS software is theoretically capable of accommodating almost any scheduling requirement, **in practice it is nearly impossible to satisfy all users simultaneously**. Resource sharing policies are fundamentally a social and organisational challenge, not a purely technical one.
 - A useful cross-reference for users migrating between different LRMS systems is the **Rosetta Stone of workload managers**, which maps equivalent commands across SLURM, PBS, LSF, and SGE.
-

0.3.4 18. Scientific Software Management

0.3.4.1 18.1 Distribution and Installation Scientific software on an HPC cluster is typically:

- Installed **cluster-wide** in a shared directory (e.g., `/opt/cluster/software` or similar).
- **Mounted read-only** on all compute nodes via NFS.
- Managed through the **module system** to handle multiple versions of the same software.

0.3.4.2 18.2 The Module System The **environment modules** system is a critical tool for managing the complexity of an HPC software stack. It allows users to **dynamically modify their shell environment** (primarily the `PATH` and `LD_LIBRARY_PATH` variables) to select specific versions of compilers, libraries, and applications without conflicting installations.

0.3.4.2.1 Core Module Commands

Command	Action
<code>module avail</code>	List all available modules on the system
<code>module list</code>	List currently loaded modules
<code>module load <name></code>	Add a module to the current environment
<code>module unload <name></code>	Remove a module from the current environment
<code>module swap <old> <new></code>	Replace one module with another
<code>module purge</code>	Unload all loaded modules
<code>module show <name></code>	Display what a module does (which variables it sets)

0.3.4.2.2 How Modules Work When a user executes `module load gcc/12.2.0`, the module system modifies key **environment variables**:

- **PATH**: prepended with the directory containing the GCC 12.2.0 executables, so that `gcc`, `g++`, and `gfortran` resolve to the correct version.
- **LD_LIBRARY_PATH**: prepended with the directory containing the GCC 12.2.0 runtime libraries, ensuring that dynamically linked executables find the correct shared libraries at runtime.

This mechanism allows multiple versions of the same software to coexist on the system without conflicts — a user can switch from GCC 11 to GCC 12 with a single `module swap` command.

0.3.5 19. Compilers in HPC

0.3.5.1 19.1 What Does Compilation Mean? **Compilation** is the process of translating human-readable **source code** (written in high-level languages such as C, C++, or Fortran) into **machine code** — a stream of binary instructions that the CPU can execute directly. In HPC, the choice of compiler and optimisation flags can have a dramatic impact on performance, sometimes yielding speedups of 2–5× for the same source code.

0.3.5.2 19.2 Available Compiler Suites Three major compiler suites are commonly available on HPC systems:

Compiler Suite	Licence	Key Characteristics
GNU (GCC)	Free / Open Source	Always available on any Linux system; many versions; broad language support (C, C++, Fortran, etc.); good general performance but sometimes lags behind commercial compilers in optimisation for specific architectures
Intel oneAPI (formerly Intel Compiler Suite)	Free (commercial support available)	A comprehensive software stack including compilers (<code>icc</code> , <code>icx</code> , <code>ifort</code> , <code>ifx</code>), MPI library (Intel MPI), profiling and benchmarking tools, and the Math Kernel Library (MKL) ; highly optimised for Intel architectures

Compiler Suite	Licence	Key Characteristics
NVIDIA HPC SDK (formerly PGI)	Free (commercial support available)	A full software stack for both CPU and GPU programming; includes compilers, libraries, profiling tools; supports CUDA Fortran and OpenACC for GPU offloading; also available for ARM architectures

0.3.5.2.1 Compilation Example with GCC A typical compilation workflow on an HPC system:

```
module load gcc/12.2.0          # Load desired compiler version
gcc -O2 -o myprogram myprogram.c # Compile with optimisation level 2
```

Common optimisation flags:

Flag	Effect
-O0	No optimisation (fastest compilation, useful for debugging)
-O1	Basic optimisations
-O2	Standard optimisation level (recommended default)
-O3	Aggressive optimisations (may increase code size; occasionally slower due to cache effects)
-Ofast	Like -O3 plus potentially unsafe math optimisations
-march=native	Optimise for the specific CPU architecture of the build machine
-S	Produce assembly output instead of an executable (useful for inspecting generated code)

0.3.6 20. Libraries: Static vs. Dynamic Linking

0.3.6.1 20.1 Overview Libraries are collections of pre-compiled functions that applications link against to avoid reimplementing common functionality (linear algebra, I/O, communication, etc.). In HPC, the choice between **static** and **dynamic** linking has significant implications for performance, portability, and maintenance.

0.3.6.2 20.2 Static Libraries Static libraries have the file extension `.a` (archive) on Linux. They are collections of **object files** (`.o`).

Characteristics: - The **linker** extracts the needed object code from the `.a` file and **embeds it directly into the output executable** at build time. - The resulting executable is **self-contained**: it has no runtime dependency on the library file. - **Advantages:** no need to locate dependencies at runtime; simpler deployment (single binary). - **Disadvantages:** can lead to **large executables** (duplicated library code in every binary that uses it); updates to the library require recompilation of all dependent programs; on modern systems, building a completely static executable is often difficult due to dependencies on dynamic system libraries (e.g., `libc`).

0.3.6.3 20.3 Dynamic (Shared) Libraries Dynamic libraries (also called **shared libraries**) have the file extension `.so` (shared object) on Linux.

Characteristics: - The executable contains only a **reference** to the shared library, not the library code itself. - At runtime, the **dynamic linker** (`ld.so / ld-linux-x86-64.so.2`) locates and loads the required shared libraries into memory. - **Advantages:** smaller executables; library updates do not require recompilation; shared memory pages (multiple processes using the same library share a single copy in RAM). - **Disadvantages:** more complex build semantics; runtime errors if libraries are not found; slight startup overhead for dynamic resolution.

0.3.6.3.1 Locating Shared Libraries at Runtime Two primary mechanisms exist for the dynamic linker to find shared libraries:

1. **LD_LIBRARY_PATH:** an environment variable listing directories to search for shared libraries. This is the mechanism managed by the **module system** on HPC clusters.
2. **RPATH:** library search paths **embedded directly in the executable** at build time (using linker flags like `-Wl,-rpath,/path/to/lib`). The executable “knows” where its dependencies live without relying on environment variables.

0.3.6.4 20.4 Diagnostic Commands Two essential Linux commands for inspecting executables and their library dependencies:

Command	Purpose	Example
<code>file <binary></code>	Identify file type, architecture, and linking mode	<code>file /usr/bin/pwd</code> → ELF 64-bit LSB pie executable, x86-64, dynamically linked
<code>ldd <binary></code>	List all shared library dependencies and their resolved paths	<code>ldd /usr/bin/pwd</code> → shows <code>libc.so.6</code> , <code>ld-linux-x86-64.so.2</code> , etc.

0.3.6.4.1 Common Runtime Error If a shared library cannot be found at runtime, execution fails immediately:

```
./xhpl: error while loading shared libraries: libmpifort.so.12:
cannot open shared object file: No such file or directory
```

This error typically indicates that the required **module has not been loaded** (i.e., LD_LIBRARY_PATH does not contain the directory holding libmpifort.so.12). The solution is:

```
module load openmpi/4.1.4    # or whichever MPI module provides the library
./xhpl
```

This example — taken from running the **HPL benchmark** on ORFEO without loading the MPI module — powerfully illustrates why the module system is essential: without it, even a correctly compiled binary cannot run.

0.3.7 Summary of Key Concepts — Lecture 3

Concept	Key Takeaway
ORFEO PNR expansion	+60% nodes, +66% RAM, +70% cores, +100% GPUs (H100); 200 Gbit/s InfiniBand added
Storage tiers	Ceph (HDD/SSD/NVMe) for performance, NFS + tape library for long-term archival
Data protection	Replica 3 for critical tiers; 6+2 erasure coding for bulk scratch
HPC software stack	Layered architecture: OS → middleware → resource manager → compilers → libraries → applications
Cluster-wide services	NTP, DNS, DHCP, NFS, SSH, LDAP, monitoring (Grafana/Zabbix)
LRMS / Batch scheduler	Manages resource allocation and job scheduling; balances user satisfaction with cluster utilisation
SLURM	Dominant open-source LRMS; entities: nodes, partitions, jobs, job steps, QoS
Module system	Dynamic environment modification (PATH, LD_LIBRARY_PATH) for managing multiple software versions
Compilers	GCC (free, universal), Intel oneAPI (optimised for Intel), NVIDIA HPC SDK (CPU+GPU)
Static vs. dynamic libraries	.a = self-contained but large; .so = smaller, shared, but requires runtime resolution
ldd / file	Essential diagnostic tools for inspecting binary format and library dependencies

0.4 Lecture 4 — Parallel Concepts and Performance Evaluation

0.4.1 21. Parallel Programming Paradigms

The choice of parallel programming paradigm in HPC is **dictated by memory organisation**. As discussed in Lecture 2, memory within a node is shared among all cores, while memory across nodes is distributed. These two hardware realities give rise to two fundamentally different programming models.

0.4.1.1 21.1 Shared Memory Programming In a **shared memory** paradigm, all processing elements (typically **threads** spawned by a single operating system process) have direct access to a **single, global address space**.

Advantages: - **Uniform global address space** provides a programmer-friendly view of memory — any thread can read or write any memory location without explicit data transfer. - **Data sharing between tasks is fast** and, in the idealised model, uniform, owing to the physical proximity of memory to CPUs.

Disadvantages: - **Does not scale to large core counts** — bus contention, cache coherency overhead, and memory bandwidth limitations impose practical upper bounds. - The programmer bears responsibility for **synchronisation constructs** (mutexes, barriers, critical sections) to ensure correct concurrent access to shared data — failure to do so leads to **race conditions** and non-deterministic bugs. - **Non-uniform memory access (NUMA)** on modern multi-socket nodes means that the “uniform” model is an abstraction; in reality, access times depend on data placement.

Standard API: OpenMP — a directive-based extension to C, C++, and Fortran. OpenMP uses compiler pragmas (e.g., `#pragma omp parallel for`) to annotate loops and code regions for parallel execution. Compilation requires a specific flag:

```
#pragma omp parallel for
for (int i = 0; i < n; ++i)
    c[i] = a[i] + b[i];
```

```
gcc -fopenmp mycode.c
```

The operating system spawns a team of threads that share the process’s address space, with work distributed across threads according to the specified schedule.

0.4.1.2 21.2 Distributed Memory Programming In a **distributed memory** paradigm, each processing element (typically an independent operating system **process**) has access **only to its own local memory**. Data exchange between processes requires explicit **message passing** over the network.

Advantages: - **Memory scales with the number of processors** — adding more nodes adds proportionally more memory, making this model suitable for problems that exceed the memory capacity of a single node.

Disadvantages: - **Data is scattered across separate address spaces** — the programmer must explicitly manage data distribution, communication patterns, and buffer management. - **Non-uniform memory access times** — data residing on a remote node takes significantly longer to access than local data (network latency vs. memory latency). - Programming complexity is higher: the programmer is responsible for **all details of inter-process data communication**.

Standard API: MPI (Message Passing Interface) — a library-based standard that defines how processes send and receive messages. Multiple implementations exist: - **OpenMPI:** widely used open-source implementation. - **Intel MPI:** optimised for Intel architectures. - **MPICH:** reference implementation.

All implementations provide a communication library and wrapper compilers (`mpicc`, `mpif90`) that handle linking against the appropriate MPI libraries. Programs are launched using a dedicated command (e.g., `mpirun` or `srun` under SLURM) that spawns processes across multiple nodes.

0.4.1.3 21.3 Mapping Paradigms to HPC Architecture A modern HPC cluster is a **distributed memory machine composed of shared memory nodes**. This architectural reality motivates the use of **hybrid programming models**:

$$\text{HPC Cluster} = \underbrace{\text{Shared Memory (OpenMP)}}_{\text{intra-node}} + \underbrace{\text{Message Passing (MPI)}}_{\text{inter-node}}$$

An important asymmetry exists between the two paradigms:

- It is **trivial to implement message passing on a shared memory machine**: each Linux process has its own private virtual address space, so MPI processes on the same node simply communicate through the operating system’s inter-process communication mechanisms (shared memory segments, pipes, etc.).
- It is **impossible to implement shared memory on a distributed memory machine**: threads are spawned by a single Linux process and inherently share that process’s address space — this cannot be replicated across physically separated nodes without hardware support (such as RDMA, which provides only a partial approximation).

0.4.1.4 21.4 Other Parallel Programming Approaches Beyond the two dominant paradigms, several additional approaches address specific hardware targets or programming convenience:

Approach	Description
MPI + OpenMP (hybrid)	Combines inter-node message passing with intra-node thread parallelism; often the most efficient model for modern clusters
CUDA	NVIDIA’s proprietary SDK for programming NVIDIA GPUs; provides fine-grained control over GPU execution
OpenCL	An open standard for heterogeneous computing across CPUs, GPUs, FPGAs, and other accelerators; “write once, run everywhere”

Approach	Description
OpenACC	A directive-based approach (similar in philosophy to OpenMP) for offloading computation to accelerators without rewriting code in proprietary languages

0.4.2 22. Fundamental Concepts in Parallel Computing

0.4.2.1 22.1 Motivation for Parallelism Parallelism in HPC serves two primary purposes:

1. **Time-to-solution reduction:**

- Solve the **same problem in less time** (strong scaling).
- Solve a **larger problem in the same time** (weak scaling).

2. **Problem size (memory capacity):**

- Solve problems whose data **cannot fit in the memory addressable by a single computational unit** — or that would suffer severe performance degradation (e.g., excessive swapping, cache thrashing) if forced into a single node’s memory.

0.4.2.2 22.2 From Serial to Parallel Execution Consider a workload consisting of 12 independent tasks, each requiring 1 second on a single processor:

- **Serial execution:** total time = 12 seconds (all tasks execute sequentially on one processor).
- **Parallel execution on 3 processors:** if the workload divides evenly, each processor handles 4 tasks, reducing total time to 4 seconds — a $3\times$ speedup.

However, this ideal scenario is rarely achieved in practice due to several complicating factors.

0.4.2.3 22.3 Load Imbalance **Load imbalance** occurs when processors cannot execute their assigned tasks at the same speed, or when the workload is not evenly divisible among processors. If one processor finishes its share early, it must **idle** until the slowest processor completes — the overall execution time is determined by the **slowest processor** (the “straggler”).

Load imbalance leads to wasted computational resources: processors that finish early contribute nothing while waiting. Achieving good **load balancing** — distributing work so that all processors finish at approximately the same time — is a fundamental challenge in parallel algorithm design.

0.4.2.4 22.4 Task Dependencies **Dependencies** between tasks constrain the degree of achievable parallelism. If task *B* depends on the output of task *A*, then *B* cannot begin execution until *A* has completed, regardless of how many processors are available. Dependencies introduce **serialisation points** that limit the speedup.

In the example above, if certain tasks have data dependencies (e.g., task 11 depends on task 7, which depends on task 3), the execution time increases from 4 seconds to 6 seconds — a 50% increase over the ideal parallel case.

0.4.2.5 22.5 The Four Principles of Parallel Computing Effective parallel computing requires mastering four interrelated principles:

Principle	Description
Finding and exploiting parallelism	Identifying which parts of an algorithm can execute concurrently
Finding and exploiting data locality	Ensuring that data is physically close to the processor that needs it, minimising costly remote memory accesses
Load balancing	Distributing work evenly across all processors to avoid idle time
Coordination and synchronisation	Managing the ordering and communication between parallel tasks to ensure correctness

These principles collectively make **parallel programming significantly more difficult than sequential programming**. A well-parallelised application must address all four simultaneously.

0.4.2.6 22.6 Illustrative Example: Parallel Sum of N Numbers The problem of computing the sum of N numbers provides a minimal but instructive example of all four parallel computing principles in action.

Serial algorithm: 1. Read N . 2. Generate N numbers and store in an array. 3. Compute the sum sequentially. 4. Write the result.

Parallel decomposition with P processors:

1. **Finding parallelism:** the summation of N numbers can be decomposed into P independent partial sums, each over N/P elements.
2. **Data locality:** each processor should hold its N/P elements in local memory to avoid remote memory accesses. In a distributed memory setting, the array must be **partitioned and distributed** across processors before computation begins.
3. **Load balancing:** perfect balance is achieved when N is evenly divisible by P (i.e., $N \bmod P = 0$). When this is not the case, a **round-robin** distribution strategy minimises imbalance — the difference in workload between the most-loaded and least-loaded processor is at most one element.
4. **Coordination and synchronisation:** after each processor computes its partial sum, the partial results must be **communicated and combined** (reduced) into a global sum. This requires an explicit coordination step — in MPI, this would be an `MPI_Reduce` operation; in OpenMP, a **reduction** clause.

The parallel algorithm becomes: 1. Read N (on one processor). 2. Distribute information about N to all P processors. 3. Each processor generates (or receives) N/P numbers. 4. Each processor computes a **partial sum**. 5. Combine all partial sums into a **global sum** (reduction). 6. Write the result (on one processor).

0.4.3 23. Parallel Performance Metrics

0.4.3.1 23.1 What Is Parallel Performance? In the context of parallel computing, **performance** refers specifically to the relationship between **computational requirements** (the work to be done) and **computational resources** (hardware, time, energy, cost) needed to meet those requirements.

The resources of interest include:

Resource	Metric
Hardware	Number of processors, cores, GPUs
Time	Wall-clock execution time
Energy	Total energy consumed (joules, kWh)
Money	Financial cost of computation (cloud billing, hardware amortisation)

0.4.3.2 23.2 Scalability **Scalability** (or **scaling**) describes how application performance changes as the “size” of the HPC infrastructure — typically measured by the number of processors P — increases. Two complementary perspectives exist:

- **Strong scaling:** can a given (fixed-size) problem be solved faster with more processors?
- **Weak scaling:** can a proportionally larger problem be solved in the same time with more processors?

Key questions that scalability analysis addresses:

1. How much faster can a given problem be solved with P workers instead of one?
2. How much more work can be done with P workers instead of one?
3. What impact do communication requirements have on performance?
4. What fraction of resources is actually used **productively** for solving the problem?
5. What are the basic limitations of a code’s implementation or algorithm for parallel processing?

0.4.3.3 23.3 Speed-Up **Speed-up** quantifies how much faster a workload executes on P processors compared to a single processor:

$$S(P) = \frac{T(1)}{T(P)}$$

where: - $T(1)$ = execution time on a single processor (the serial baseline), - $T(P)$ = execution time on P processors.

Interpretation: - $S(P) = P$ corresponds to **perfect (linear) speed-up** — the ideal case where doubling the processors halves the execution time. - $S(P) < P$ indicates **sub-linear speed-up** — the common case, due to serial fractions, communication overhead, and load imbalance. - $S(P) > P$ indicates **super-linear speed-up** — a seemingly paradoxical result that can arise from: - **Cache effects:** distributing the problem across more processors may allow each processor’s share to fit

in fast cache memory, dramatically reducing memory access time. - **Poor baseline:** if $T(1)$ is measured using an unoptimised serial code while the parallel code benefits from additional optimisations. - **Search problems:** in some algorithms (e.g., branch-and-bound), parallel exploration of the search space may find the solution faster by exploring different branches simultaneously.

0.4.3.4 23.4 Efficiency **Parallel efficiency** measures how productively the processors are utilised on average:

$$E(P) = \frac{S(P)}{P} = \frac{T(1)}{P \cdot T(P)}$$

Interpretation: - $E(P) = 1$ (100%) corresponds to perfect efficiency — every processor is fully utilised for productive computation. - $E(P) < 1$ indicates that some fraction of processor time is wasted on communication, synchronisation, idle waiting, or redundant computation.

Important caveat: Speed-up and efficiency are **relative metrics** — they compare an application’s performance on P processors against its performance on 1 processor. They are **not absolute performance metrics**. A more rigorous comparison uses the **best available serial algorithm** on 1 processor as the baseline, rather than the parallel algorithm running on 1 processor (which may include parallel overhead even when $P = 1$).

Understanding **why** an application fails to achieve linear scaling is the key to improving its performance on parallel computers.

0.4.3.5 23.5 Formal Notation The following notation is used consistently throughout the performance analysis:

Symbol	Meaning
n	Problem size
$T_S(n)$	Serial run-time for a problem of size n
$T_P(n)$	Parallel run-time with P processes for a problem of size n
P	Number of computing units (processors, cores)
f_n	Intrinsic sequential fraction of the problem of size n
$k(n, t)$	Parallel overhead (communication, synchronisation)

0.4.3.6 23.6 Naïve Expectations vs. Reality Beginners often hold the following (incorrect) assumptions about parallel performance:

Naïve Expectation	Reality
If one processor delivers m MFLOPS, then P processors deliver $P \times m$ MFLOPS	Communication overhead, serial fractions, and memory bandwidth contention reduce effective throughput
If serial run-time is T , parallel run-time is T/P	Only true if the entire workload is perfectly parallelisable with zero overhead
Doubling processors halves execution time	Diminishing returns due to Amdahl's Law
Doubling problem size doubles execution time	Depends on algorithmic complexity; many algorithms have super-linear scaling with problem size

These expectations serve as useful idealisations but are never fully realised in practice.

0.4.4 24. Amdahl's Law

0.4.4.1 24.1 Assumptions and Derivation **Amdahl's Law** (Gene Amdahl, 1967) provides an upper bound on the speed-up achievable by parallelisation, given that a fraction of the workload is inherently sequential.

Assumptions: - The underlying hardware is perfectly scalable (no hardware bottlenecks). - The total workload consists of a **serial fraction** s and a **parallel fraction** p , with $s + p = 1$. - The serial baseline is normalised: $T(1) = s + p = 1$. - The parallel fraction scales perfectly with P processors.

Derivation:

The parallel execution time is:

$$T(P) = s + \frac{p}{P}$$

The serial fraction s cannot be reduced regardless of how many processors are used, while the parallel fraction p is divided equally among P processors. The speed-up is therefore:

$$S(P) = \frac{T(1)}{T(P)} = \frac{1}{s + \frac{p}{P}} = \frac{1}{s + \frac{1-s}{P}}$$

0.4.4.2 24.2 The Fundamental Limit Taking the limit as $P \rightarrow \infty$:

$$\lim_{P \rightarrow \infty} S(P) = \frac{1}{s}$$

This is the central result of Amdahl's Law: **the maximum achievable speed-up is bounded by the reciprocal of the serial fraction**, regardless of the number of processors.

Practical implications:

Serial Fraction s	Maximum Speed-Up $1/s$
50%	$2\times$
10%	$10\times$
5%	$20\times$
1%	$100\times$

Even a seemingly small serial fraction of 5% limits the maximum speed-up to $20\times$, regardless of whether 100, 1,000, or 1,000,000 processors are used. This result powerfully motivates the effort to **minimise the serial fraction** of any parallel application.

0.4.4.3 24.3 Amdahl's Law with Communication Overhead In practice, parallel execution incurs **communication overhead** $c(P)$ — the time spent on inter-process communication, synchronisation, and coordination. Incorporating this overhead:

$$T(P) = s + \frac{p}{P} + c(P)$$

The speed-up becomes:

$$S(P) = \frac{1}{s + \frac{1-s}{P} + c(P)}$$

The form of $c(P)$ depends on the communication pattern and network topology:

Communication Pattern	Scaling	Example
Global collective (barrier, broadcast, reduction)	$c(P) = k \log P$	Tree-based reduction across P processors
All-to-one or serialised communication	$c(P) = kP$	Every process sends a message over a shared bus; sequential message handling

0.4.4.3.1 Worked Example: Linear Communication Overhead Consider a simple communication model where each processor incurs a constant communication fraction k per interaction. With $P = 4$ processors:

$$T(4) = s + \frac{p}{4} + 4k$$

The speed-up becomes:

$$S(P) = \frac{1}{s + \frac{1-s}{P} + kP}$$

0.4.4.4 24.4 Behaviour at Large P Examining the asymptotic behaviour:

- **Pure Amdahl's Law** ($k = 0$): $S \rightarrow 1/s$ as $P \rightarrow \infty$ — speed-up saturates at a finite value independent of P .
- **With communication overhead** ($k > 0$): $S \rightarrow 1/(kP)$ as $P \rightarrow \infty$ — speed-up not only saturates but **decreases** for sufficiently large P , because the communication overhead eventually dominates.

This means there exists an **optimal number of processors** P^* beyond which adding more processors actually **slows down** the computation. Finding this optimum requires balancing the diminishing returns of parallelisation against the growing communication cost.

The existence of a speed-up maximum followed by degradation is a common observation in HPC practice and underscores the importance of **communication-efficient algorithms** and **high-speed interconnects**.

0.4.5 25. Gustafson's Law

0.4.5.1 25.1 Motivation: The Weak Scaling Perspective Amdahl's Law assumes a **fixed problem size** — as more processors are added, the same amount of work is distributed more finely. However, in many practical scenarios, users do not simply want to solve the same problem faster; they want to solve **bigger problems** using more resources. This is the **weak scaling** perspective.

A critical observation motivates this shift: **the proportion of computation that is sequential normally decreases as the problem size increases**. In other words, the serial fraction s is not a fixed constant — it typically shrinks as the problem grows, because the serial components (initialisation, I/O, global reductions) remain constant while the parallel components (the main computational loops) grow with problem size.

0.4.5.2 25.2 Derivation Gustafson's Law (John Gustafson, 1988) adopts the following assumptions:

- The total execution time on P processors is held **constant** (normalised to 1).
- As P increases, the **parallel workload increases linearly** with P : $p \rightarrow Pp$.
- The serial fraction s remains constant.

The parallel execution time is:

$$T(P) = s + \frac{Pp}{P} = s + p = 1$$

This confirms the normalisation: execution time remains constant regardless of P .

Now, ask: **how long would it take to solve this (enlarged) workload on a single processor?**

$$T_P(1) = s + Pp$$

The **scaled speed-up** is:

$$S(P) = \frac{T_P(1)}{T(P)} = \frac{s + Pp}{s + p} = s + Pp = s + P(1 - s)$$

Simplifying:

$$S(P) = P - (P - 1)s$$

0.4.5.3 25.3 Interpretation Gustafson’s Law yields a fundamentally more optimistic view of parallel scalability:

- The speed-up is a **linear function of P** , with slope $(1 - s)$.
- As P increases, speed-up grows **without bound** (unlike Amdahl’s Law, which saturates at $1/s$).
- The serial fraction s merely reduces the slope from perfect linearity but does not impose an absolute ceiling.

Comparison of the two laws:

Property	Amdahl’s Law	Gustafson’s Law
Perspective	Fixed problem size (strong scaling)	Scaled problem size (weak scaling)
Question answered	How much faster can I solve <i>this</i> problem?	How much <i>more</i> work can I do in the same time?
Speed-up behaviour	Saturates at $1/s$	Grows linearly as $P - (P - 1)s$
Practical relevance	Pessimistic bound; highlights serial bottlenecks	Optimistic bound; reflects real usage patterns in scientific computing

Both laws are correct descriptions of different scaling scenarios. The choice of which is more relevant depends on the user’s intent: if the problem size is fixed, Amdahl’s Law governs; if the problem size grows with available resources (as is typical in scientific simulation), Gustafson’s Law provides the more relevant prediction.

0.4.6 26. Sustained Performance in Practice

Despite the theoretical availability of petaflops or exaflops of peak performance, **real scientific applications** achieve only a fraction of the theoretical peak. Historical data from production HPC systems illustrate this gap vividly.

For example, on the **Blue Waters** system at NCSA (22,640 AMD 6276 processors, theoretical peak of 13 PFLOPS), sustained performance on real scientific codes falls significantly below the theoretical peak — a pattern consistent across virtually all large-scale HPC systems.

This performance gap arises from the interaction of four fundamental degradation factors.

0.4.7 27. Sources of Performance Degradation

At an abstract level, four factors prevent an HPC system from fully exploiting all its resources all of the time:

0.4.7.1 27.1 Starvation **Starvation** occurs when there is **insufficient work available** to keep all processing units busy at every instant. This manifests as idle functional units waiting for work to be assigned.

Typical causes: - Insufficient **parallel work** for the available number of processors — the problem is too small relative to the machine size. - Parallel work is **unevenly distributed** among processors (**load imbalance**), causing some processors to complete their share early and idle while waiting for stragglers.

0.4.7.2 27.2 Latency **Latency** is the **time delay** for information to travel from one part of the system to another.

Typical causes: - **Memory access latency:** the time for data to travel from DRAM to the CPU (typically 50–100 ns for local DRAM, 100–300 ns for remote NUMA access). - **Network latency:** the time for a message to travel between nodes (typically 1–5 μ s for InfiniBand, 10–50 μ s for Ethernet).

Modern architectures employ various techniques to **hide latency**, including prefetching, out-of-order execution, multithreading, and communication-computation overlap. These will be discussed in subsequent lectures.

0.4.7.3 27.3 Overhead **Overhead** encompasses all **additional work** that is necessary for parallelisation but does not directly contribute to solving the computational problem.

Typical causes: - **Thread/process creation and destruction** (spawning and joining parallel tasks). - **Synchronisation** (barriers, locks, critical sections). - **Communication setup** (establishing connections, serialising/deserialising messages). - **Bookkeeping** (managing parallel data structures, scheduling decisions).

These operations **steal resources** from productive computation and should be minimised through careful algorithm and system design.

0.4.7.4 27.4 Contention **Contention** occurs when two or more tasks simultaneously request access to the **same resource** (hardware or software).

Typical causes: - Multiple tasks writing to the **same disk** simultaneously. - Multiple tasks sending messages to the **same memory location** or network port at the same time. - **Lock contention:** multiple threads waiting to acquire the same mutex.

Contention events are often **unpredictable** and therefore difficult to avoid or optimise. They introduce **variable and non-deterministic delays** that degrade both performance and reproducibility.

0.4.7.5 27.5 From Ideal to Reality: A Visual Summary The progression from idealised to realistic parallel execution can be summarised in three stages:

1. **Ideal world:** all work is perfectly parallelisable with zero overhead — speed-up equals P .
2. **First correction (Amdahl):** serial portions of the code limit maximum speed-up to $1/s$.
3. **Full reality:** communication overhead, synchronisation costs, load imbalance, and contention further reduce the achieved speed-up, often dramatically, below even the Amdahl bound.

Understanding and quantifying these degradation sources — through profiling, tracing, and performance modelling — is the central challenge of **HPC performance engineering**.

0.4.8 Summary of Key Concepts — Lecture 4

Concept	Key Takeaway
Shared memory paradigm	Threads share a global address space; simple programming model but limited scalability; standard: OpenMP
Distributed memory paradigm	Processes have private memory; explicit message passing required; scales to thousands of nodes; standard: MPI
Hybrid programming	MPI (inter-node) + OpenMP (intra-node) is the dominant model for modern HPC clusters
Load balancing	Uneven work distribution wastes resources; execution time is bounded by the slowest processor
Task dependencies	Serialisation points limit achievable parallelism regardless of processor count
Speed-up $S(P)$	Ratio of serial to parallel execution time; $S(P) = P$ is the ideal (linear) case
Efficiency $E(P)$	$S(P)/P$; measures productive utilisation of processors; ideal value is 1
Amdahl's Law	$S(P) = 1/(s + (1 - s)/P)$; maximum speed-up bounded by $1/s$; governs strong scaling
Amdahl + communication	$S(P) = 1/(s + (1 - s)/P + kP)$; speed-up degrades beyond an optimal P^*
Gustafson's Law	$S(P) = P - (P - 1)s$; speed-up grows linearly with P under weak scaling
Starvation	Insufficient or unevenly distributed parallel work
Latency	Time delay for data movement (memory, network)
Overhead	Non-productive work required for parallelisation (spawning, synchronisation)

Concept	Key Takeaway
Contention	Simultaneous access to shared resources causing unpredictable delays

0.5 Lecture 5 — Message Passing Programming with MPI

0.5.1 28. The Message Passing Interface (MPI)

0.5.1.1 28.1 Recap: The Message Passing Paradigm As introduced in Lecture 4 (Section 21.2), the message passing paradigm operates under a **shared-nothing** model: each parallel process possesses its own **private address space**, and no process can directly read or write another process's memory. Data exchange occurs exclusively through **explicit send and receive operations**. The programmer bears full responsibility for:

- **Memory motion:** deciding when and what data must be communicated.
- **Data distribution:** partitioning problem data across processes.
- **Communication patterns:** structuring the sequence of sends and receives to avoid errors such as deadlock.

A concrete illustration of the shared-nothing principle: if process 0 declares `A(10)` and process 1 also declares `A(10)`, these are **entirely different variables** residing in separate address spaces. Modifying `A(10)` in process 0 has no effect on `A(10)` in process 1 — any synchronisation of values requires an explicit message.

0.5.1.2 28.2 What Is MPI? MPI (Message Passing Interface) is a *de facto* standard specification for message passing libraries. Key characteristics:

- **Standard by consensus:** MPI was designed through an open forum involving over **40 organisations** — hardware vendors, researchers, academics, software library developers, and end users. This broad participation ensured rapid and widespread adoption.
- **Not a formal standard:** MPI was not issued by a standards body such as ANSI or ISO. It is a community-driven specification maintained by the **MPI Forum** (www.mpi-forum.org).
- **Specification, not implementation:** the MPI document defines the **interface** (function signatures, semantics, guarantees). Actual implementations are developed independently by different organisations and optimised for specific hardware.

0.5.1.3 28.3 MPI Implementations Since MPI is a specification, multiple implementations exist:

Implementation	Notes
MPICH	The original reference implementation; basis for many vendor-specific implementations
OpenMPI	Widely used open-source implementation; popular on academic and research clusters
Intel MPI	Optimised for Intel architectures; part of the Intel oneAPI toolkit

All implementations provide: - A **communication library** linked against by user programs. - **Wrapper compilers** (`mpicc` for C, `mpicxx` for C++, `mpif90` for Fortran) that automatically handle include paths and library linking. - A **launcher** (`mpirun` or `mpiexec`) for starting parallel programs across multiple processes and nodes.

0.5.1.4 28.4 MPI Version History The MPI standard has evolved significantly since its inception:

Version	Year	Key Developments
MPI-1.0	1994	Initial standard: point-to-point communication, collective operations, communicators, datatypes
MPI-1.1	1995	Minor corrections and clarifications
MPI-2.0	1997	Parallel I/O (MPI-IO), one-sided communication (RMA), dynamic process management
MPI-3.0	2012	Non-blocking collectives, neighbourhood collectives, improved one-sided communication
MPI-3.1	2015	Small additions and corrections to MPI-3.0; most widely deployed version
MPI-4.1	2023	Large-count support, partitioned communication, persistent collectives
MPI-5.0	2025	Latest approved version (June 2025)

MPI-3.x remains the most widely implemented version across both Intel MPI and OpenMPI, though support for MPI-4.x features is progressively expanding.

0.5.1.5 28.5 Reasons to Use MPI Several factors make MPI the dominant parallel programming model in HPC:

1. **International standard:** ensures code portability across platforms and vendors.
2. **Continuous evolution:** the standard adapts to new hardware capabilities and programming patterns.
3. **Universal availability:** free implementations (MPICH, OpenMPI) run on virtually all parallel systems, with interfaces for **C/C++** and **Fortran**.
4. **Rich functionality:** provides optimised communication primitives for a wide range of patterns (point-to-point, collective, one-sided).
5. **Hardware flexibility:** works on both **distributed memory** and **shared memory** architectures.
6. **Scalability:** supports large-scale program development and modular integration.

0.5.2 29. Programming with MPI: Fundamentals

0.5.2.1 29.1 MPI as a Library MPI is implemented as a **library**, not a language extension. All MPI operations are performed through **function calls** (C) or **subroutine calls** (Fortran). The required header files are:

Language	Header
C / C++	<code>#include <mpi.h></code>
Fortran 77/90	<code>include 'mpif.h'</code>
Fortran 90 (preferred)	<code>use mpi</code> (module-based, optional but recommended)

0.5.2.2 29.2 Compiling MPI Programs There is **no standard compilation procedure** defined by the MPI specification — it is left to each implementation. In practice, two approaches exist:

1. **Manual linking:** specify include directories and libraries explicitly:

```
gcc -I/mpidir/include -L/mpidir/lib -lmpi myprogram.c -o myprogram
```

2. **Wrapper compilers** (recommended): use implementation-provided wrappers that handle all linking details automatically:

```
mpicc myprogram.c -o myprogram      # C
mpicxx myprogram.cpp -o myprogram   # C++
mpif90 myprogram.f90 -o myprogram   # Fortran
```

0.5.2.3 29.3 Running MPI Programs The MPI standard **does not specify** how to launch an MPI program — this is implementation- and system-dependent. Common approaches:

- **Direct launch:** `mpirun -np <N> ./myprogram` (provided by most MPI implementations).
- **SLURM integration:** `srun ./myprogram` (when running under the SLURM resource manager, as on ORFEO). SLURM's `srun` automatically uses the job's allocated resources to distribute MPI processes across nodes.

0.5.2.4 29.4 Categories of MPI Routines MPI routines fall into four broad classes:

Class	Purpose	Examples
Environment management	Initialise, query, and finalise the MPI runtime	MPI_Init, MPI_Comm_size, MPI_Comm_rank, MPI_Finalize
Point-to-point communication	Send and receive data between pairs of processes	MPI_Send, MPI_Recv, MPI_Isend, MPI_Irecv
Collective communication	Coordinate and exchange data among groups of processes	MPI_Bcast, MPI_Reduce, MPI_Alltoall, MPI_Barrier
Derived datatypes	Construct custom data layouts for communication	MPI_Type_contiguous, MPI_Type_vector, MPI_Type_struct

0.5.2.5 29.5 The Minimal MPI Program The **six essential routines** that constitute the minimal MPI toolkit are:

Routine	Purpose
MPI_Init	Initialise the MPI execution environment
MPI_Comm_size	Query the total number of processes in a communicator
MPI_Comm_rank	Query the rank (unique identifier) of the calling process
MPI_Send	Send a message to another process
MPI_Recv	Receive a message from another process
MPI_Finalize	Terminate the MPI execution environment

With these six calls, virtually any message-passing algorithm can be expressed.

0.5.3 30. MPI Initialisation and Environment

0.5.3.1 30.1 Basic Initialisation The simplest possible MPI program initialises and finalises the MPI environment:

```
#include <mpi.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    MPI_Finalize();
    return 0;
}
```

Key rules: - `MPI_Init` must be called **before any other MPI routine**. Until initialisation, all processes execute independently with no MPI coordination. - `MPI_Finalize` should be called at the end of the program. After finalisation, no further MPI calls are permitted. - **Best practice:** always explicitly return from `main` with a value, and always call `MPI_Finalize` before exiting.

0.5.3.2 30.2 Thread-Safe Initialisation: `MPI_Init_thread` For programs that may use **multithreading** (e.g., hybrid MPI+OpenMP), the preferred initialisation routine is `MPI_Init_thread`, which negotiates the level of thread support with the MPI implementation:

```
int mpi_provided_thread_level;
MPI_Init_thread(&argc, &argv, REQUIRED_LEVEL, &mpi_provided_thread_level);

if (mpi_provided_thread_level < REQUIRED_LEVEL) {
    // Handle insufficient thread support (e.g., abort or fall back)
}
```

The four thread-support levels, in increasing order of capability, are:

Level	Meaning
<code>MPI_THREAD_SINGLE</code>	Only one thread will execute (no multithreading)
<code>MPI_THREAD_FUNNELED</code>	Multiple threads may exist, but only the thread that called <code>MPI_Init_thread</code> will make MPI calls
<code>MPI_THREAD_SERIALIZED</code>	Multiple threads may make MPI calls, but only one at a time (serialised access)
<code>MPI_THREAD_MULTIPLE</code>	Any thread may call MPI at any time with no restrictions (full thread safety)

The implementation may provide a **lower level** than requested. It is the programmer's responsibility to check `mpi_provided_thread_level` and handle the discrepancy. Even for programs that do not use threads, calling `MPI_Init_thread` with `MPI_THREAD_SINGLE` is considered **best practice** for forward compatibility.

0.5.3.3 30.3 Querying the Environment: Size and Rank After initialisation, two fundamental queries establish the parallel context:

```
int Ntasks, Myrank;
MPI_Comm_size(MPI_COMM_WORLD, &Ntasks); // Total number of processes
MPI_Comm_rank(MPI_COMM_WORLD, &Myrank); // This process's unique ID
```

- **Size:** the total number of processes in the communicator (P). Determined at launch time (e.g., `mpirun -np 8` sets `size = 8`).

- **Rank:** an integer in the range $[0, P - 1]$ that uniquely identifies each process within the communicator. Ranks are used as **source** and **destination** addresses in all communication calls.

The canonical “Hello World” MPI program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int Ntasks, Myrank;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_SINGLE, &provided);
    MPI_Comm_size(MPI_COMM_WORLD, &Ntasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &Myrank);
    printf("Hello MPI world from task %d out of %d\n", Myrank, Ntasks);
    MPI_Finalize();
    return 0;
}
```

Important observations on I/O in MPI: - Each process executes `printf` independently — there is no coordination of output. - **Output order is undefined:** lines may be interleaved by character, by line, or by blocks. The MPI standard does not specify I/O ordering for standard output. - Parallel I/O was not part of MPI-1; **MPI-IO** was introduced in MPI-2 for coordinated file access.

0.5.3.4 30.4 C Binding Conventions All MPI C functions follow consistent naming and return conventions:

- **Naming:** `MPI_Xxxxx` — MPI is fully capitalised, the first letter after the underscore is capitalised, and all subsequent letters are lowercase. Case sensitivity matters.
- **Return value:** all MPI C functions return an integer **error code**. If the return value equals `MPI_SUCCESS`, the call completed successfully.
- **Header:** C programs must include `mpi.h`, which defines all MPI functions, constants (e.g., `MPI_SUCCESS`, `MPI_COMM_WORLD`), and types.

0.5.4 31. MPI Communicators

0.5.4.1 31.1 Concept and Purpose A **communicator** is an MPI object that defines a **group of processes** and a **communication context** within which those processes can exchange messages. Every MPI communication call requires a communicator argument, which specifies the scope of the operation.

Formally:

$$\text{Communicator} = \text{Group} + \text{Context}$$

- **Group:** a set of MPI processes, each identified by a rank within the group.
- **Context:** an internal tag that distinguishes messages sent in different communicators, even if the underlying group is the same.

0.5.4.2 31.2 Predefined Communicators MPI provides three predefined communicators:

Communicator	Description
<code>MPI_COMM_WORLD</code>	The default communicator, available immediately after <code>MPI_Init</code> . Its group contains all processes launched by the job
<code>MPI_COMM_SELF</code>	Contains only the calling process itself (group of size 1)
<code>MPI_COMM_NULL</code>	Signals an invalid or non-existent communicator

Additionally, `MPI_GROUP_NULL` signals an invalid or non-existent group.

0.5.4.3 31.3 Groups and Multiple Communicators MPI's communicator mechanism supports sophisticated process organisation:

- **A process can belong to multiple groups simultaneously** — analogous to a person being part of both a work team and a social group.
- **The same group can be associated with different communicators**, creating distinct communication contexts. A process's rank may differ across communicators (e.g., a process might be rank 3 in `MPI_COMM_WORLD` but rank 0 in a sub-communicator).
- **Groups may overlap** — the same process can appear in multiple groups.

0.5.4.4 31.4 Why Use Custom Communicators? Custom communicators serve two critical purposes:

1. **Context isolation:** prevents message interference between different modules or libraries. Without separate communicators, a library's internal `MPI_Isend/MPI_Irecv` calls could accidentally match with the application's sends and receives, leading to data corruption or deadlock.
2. **Expressing hierarchical parallelism:** by partitioning processes into sub-groups (e.g., one group per compute node, or one group per physical domain in a simulation), the programmer can express multiple levels of parallelism cleanly.

Best practice — always create a duplicate communicator for your application:

```
MPI_Comm myCOMM;
MPI_Comm_dup(MPI_COMM_WORLD, &myCOMM);
```


`MPI_Comm_dup` creates a new communicator with the same group as the original but a **new, distinct context**. This ensures that the application's messages are completely isolated from any library or module that also uses MPI internally. As Victor Eijkhout explains: without separate communicators, non-blocking calls in a library (e.g., `MPI_Isend/MPI_Irecv` in one routine, `MPI_Wait` in another) could be inadvertently matched by user code executing in between, leading to subtle and difficult-to-diagnose errors.

0.5.5 32. Point-to-Point Communication

0.5.5.1 32.1 Anatomy of an MPI Message An MPI message, like a physical letter, consists of:

Component	MPI Equivalent
Content (the letter)	Data buffer: described by a triple (address , count , datatype)
Destination address	Destination rank within the communicator
Sender address	Source rank within the communicator
Envelope hint	Integer tag for message identification
Postal service	The communicator (defines the group and context)

A send/receive operation is thus fully specified by answering four questions:

1. **How will data be described?** → MPI **datatypes**
2. **How will processes be identified?** → **rank** within a **communicator**
3. **How will the receiver recognise messages?** → integer **tag**
4. **What does completion mean?** → **blocking** vs. **non-blocking** semantics

0.5.5.2 32.2 MPI Datatypes MPI defines its own type system to ensure portability across heterogeneous architectures (where, for example, `int` might be 32 bits on one machine and 64 bits on another). The data in a message is described by a **triple** (*address, count, datatype*):

- **address**: pointer to the start of the data buffer.
- **count**: number of elements of the specified type.
- **datatype**: an MPI datatype descriptor.

MPI datatypes can be: - **Predefined**: corresponding to standard language types. - **Derived**: user-constructed types for non-contiguous data (strided blocks, indexed arrays, arbitrary structures). MPI provides functions such as `MPI_Type_contiguous`, `MPI_Type_vector`, and `MPI_Type_create_struct` for constructing derived types, enabling efficient communication of complex data layouts (e.g., sub-arrays, struct fields) without manual packing.

Predefined MPI datatypes for C:

MPI Datatype	C Datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	(raw byte)
MPI_PACKED	(for manually packed data)

Predefined MPI datatypes for Fortran:

MPI Datatype	Fortran Datatype
MPI_INTEGER	INTEGER
MPI_REAL	REAL
MPI_DOUBLE_PRECISION	DOUBLE PRECISION
MPI_COMPLEX	COMPLEX
MPI_DOUBLE_COMPLEX	DOUBLE COMPLEX
MPI_LOGICAL	LOGICAL
MPI_CHARACTER	CHARACTER(1)

0.5.5.3 32.3 Message Tags Each message carries an integer **tag** that allows the receiver to **discriminate** among multiple incoming messages from the same source:

- The sender assigns a tag to each outgoing message.
- The receiver can specify a specific tag to **screen** (filter) incoming messages.
- Alternatively, the receiver can specify MPI_ANY_TAG to accept a message with **any tag**.

Tags are essential when a process expects multiple distinct messages from the same source (e.g., one message containing coordinates and another containing velocities).

0.5.5.4 32.4 MPI_Send The basic **blocking send** operation:

```
int MPI_Send(const void *buffer, int count, MPI_Datatype datatype,
             int destination, int tag, MPI_Comm communicator);
```

Parameter	Description
buffer	Address of the data to send
count	Number of elements in the buffer
datatype	MPI datatype of each element
destination	Rank of the receiving process
tag	Integer message tag
communicator	Communicator defining the process group

0.5.5.5 32.5 MPI_Recv The basic **blocking receive** operation:

```
int MPI_Recv(void *buffer, int count, MPI_Datatype datatype,
            int source, int tag, MPI_Comm communicator,
            MPI_Status *status);
```

The receive differs from the send in two important ways:

1. **Wildcards:** the `source` parameter can be set to `MPI_ANY_SOURCE` (accept from any sender), and `tag` can be set to `MPI_ANY_TAG` (accept any tag). These wildcards provide flexibility but require subsequent inspection of the `status` object to determine the actual source and tag.
2. **Status object:** an output parameter that provides metadata about the received message:

```
int recvd_tag, recvd_from, recvd_count;
MPI_Status status;
MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, ..., &status);
recvd_tag = status.MPI_TAG;           // Actual tag of the received message
recvd_from = status.MPI_SOURCE;       // Actual source rank
MPI_Get_count(&status, datatype, &recvd_count); // Number of elements received
```

0.5.5.6 32.6 Complete Example: Two-Process Communication A minimal Fortran example demonstrating send and receive between two processes:

```
Program MPI
  Implicit None
  Include 'mpif.h'
  Integer :: rank, buffer, error
  Integer, Dimension(MPI_STATUS_SIZE) :: status

  Call MPI_Init(error)
  Call MPI_Comm_rank(MPI_COMM_WORLD, rank, error)

  If (rank == 0) Then
    buffer = 33
    Call MPI_Send(buffer, 1, MPI_INTEGER, 1, 10, MPI_COMM_WORLD, error)
  End If
```

```

If (rank == 1) Then
    Call MPI_Recv(buffer, 1, MPI_INTEGER, 0, 10, MPI_COMM_WORLD, status, error)
    Print *, 'Rank', rank, 'buffer=', buffer
End If

Call MPI_Finalize(error)
End Program

```

Key observations: - This program requires **exactly 2 processes** to function correctly (ranks 0 and 1). - It *can* be run on more than 2 processes — the additional processes simply execute no communication and proceed to `MPI_Finalize`. However, running on fewer than 2 processes would leave process 0's send without a matching receive. - The **tag** (10) must match between the send and receive calls.

0.5.6 33. Blocking vs. Non-Blocking Communication

0.5.6.1 33.1 Completion Semantics The distinction between blocking and non-blocking communication rests on the definition of **completion**:

- A **send** is complete when it is **safe to modify the send buffer** — the data has either been delivered to the destination or copied to an internal system buffer.
- A **receive** is complete when it is **safe to access the received data** — the data has been fully written into the user's receive buffer.

0.5.6.2 33.2 Blocking Communication **Blocking** calls do not return control to the caller until the operation is complete:

- `MPI_Send` blocks until the send buffer can be safely reused.
- `MPI_Recv` blocks until the message has been fully received into the buffer.

`MPI_Send` and `MPI_Recv` are the standard blocking operations.

0.5.6.3 33.3 Non-Blocking Communication **Non-blocking** calls initiate the communication and return **immediately**, allowing the program to continue execution while the data transfer proceeds in the background:

- The programmer **must not modify** (for sends) or **access** (for receives) the buffer until the operation has been verified as complete.
- Completion is checked using `MPI_Wait` (blocking check) or `MPI_Test` (non-blocking check).

Purpose: non-blocking communication enables **latency hiding** — the program can perform useful computation while data is being transferred, overlapping communication with computation.

0.5.6.4 33.4 Communication Modes Overview MPI defines multiple **send modes**, each with different completion conditions. Both blocking and non-blocking variants exist for every mode:

Mode	Completion Condition	Blocking	Non-Blocking
Standard send	Message sent (receiver state unknown)	<code>MPI_Send</code>	<code>MPI_Isend</code>
Receive	Message has arrived and is available	<code>MPI_Recv</code>	<code>MPI_Irecv</code>
Synchronous send	Completes only when the matching receive has been posted	<code>MPI_Ssend</code>	<code>MPI_Issend</code>
Buffered send	Always completes immediately (data copied to user-supplied buffer)	<code>MPI_Bsend</code>	<code>MPI_Ibsend</code>
Ready send	Assumes matching receive already posted; undefined behaviour otherwise	<code>MPI_Rsend</code>	<code>MPI_Irsend</code>

0.5.6.5 33.5 Message Overhead The time a process spends waiting for a communication call to return consists of two components:

$$\text{Total overhead} = \text{System overhead} + \text{Synchronisation overhead}$$

- **System overhead:** time spent transferring buffer contents (memory copies, network operations).
- **Synchronisation overhead:** time spent waiting for the partner process (e.g., waiting for a matching receive to be posted).

Each communication mode offers a different trade-off between these two overhead sources.

0.5.7 34. Detailed Send Modes

0.5.7.1 34.1 Synchronous Send (`MPI_Ssend`) The **safest** point-to-point communication mode. The send does not complete until the receiving process has posted a matching receive and signalled readiness:

- **Guarantees:** the receiver is ready before data transfer begins — eliminates the possibility of buffer overflow on the receiver side.
- **Cost:** highest synchronisation overhead, as the sender may idle waiting for the receiver.
- **Use case:** correctness-critical applications where message ordering and buffer safety are paramount.

0.5.7.2 34.2 Buffered Send (MPI_Bsend) The send copies data from the message buffer to a **user-supplied buffer**, then returns immediately. The actual network transfer occurs later, once the receiver posts its matching receive.

Implications: 1. The timing of the corresponding receive is irrelevant to the sender — no synchronisation wait. 2. The original send buffer can be modified immediately after the call returns. 3. **Synchronisation overhead is eliminated** on the sender side. 4. **System overhead is added:** an extra memory copy into the user-supplied buffer. 5. Requires the user to explicitly attach buffer space via `MPI_Buffer_attach`.

Trade-off: trades memory (for the buffer) and an extra copy for reduced synchronisation delay. Not suitable for very large messages, where the buffer copy becomes expensive or may exhaust available memory.

0.5.7.3 34.3 Ready Send (MPI_Rsend) Reduces both system and synchronisation overhead by **assuming** that the matching receive has **already been posted** when the send begins. Conceptually, the send simply pushes data to the network without handshaking.

- If the matching receive **has** been posted: the send completes efficiently with minimal overhead.
- If the matching receive **has not** been posted: **an error is generated** (undefined behaviour in many implementations).

Use case: rare; only appropriate when the programmer can guarantee (through program logic or synchronisation) that the receiver is already waiting.

0.5.7.4 34.4 Standard Send (MPI_Send) The **default** send mode — and the most difficult to characterise precisely, because its behaviour is **implementation-defined**:

- The MPI specification intentionally leaves the implementation free to choose the most efficient strategy based on the system’s capabilities.
- In practice, most implementations use a **dual strategy** based on message size:

Message Size	Behaviour
Small messages (below a threshold)	Data is copied to a system-supplied buffer (eager protocol). The send returns immediately, regardless of whether the receiver has posted a matching receive. Effectively behaves like a buffered send.
Large messages (above the threshold)	The send blocks until the receiver posts a matching receive (rendezvous protocol). Effectively behaves like a synchronous send.

The threshold between these two regimes is implementation-specific. For Intel MPI, it can be controlled via the environment variable `I_MPI_EAGER_THRESHOLD`.

This dual behaviour explains a common source of confusion: `MPI_Send` may or may not block depending on the message size, the MPI implementation, and the system’s buffering capacity. Programs that rely on `MPI_Send` not blocking (for small messages) are **non-portable** and may deadlock on different systems.

0.5.7.5 34.5 Choosing a Communication Mode The recommended strategy, in order of preference:

1. **Design for `MPI_Ssend`:** write the program as if all sends are synchronous — this is the safest approach and eliminates reliance on implementation-specific buffering. If the program is correct with `MPI_Ssend`, it will be correct with any send mode.
 2. **Use `MPI_Send` in practice:** this gives the MPI implementation maximum flexibility to optimise data delivery (using eager protocol for small messages, rendezvous for large ones).
 3. **Use `MPI_Isend`/`MPI_Irecv` when non-blocking is needed:** if computation can overlap with communication, non-blocking calls provide latency-hiding opportunities.
 4. **Use `MPI_Bsend` as a last resort:** only when restructuring the code to use `MPI_Isend` is too inconvenient.
 5. **Avoid `MPI_Rsend`** except in system-level, platform-specific code: the precondition (receiver must have already posted the receive) is fragile and hard to guarantee in general.
-

0.5.8 35. Non-Blocking Communication in Detail

0.5.8.1 35.1 Syntax and Request Handles Non-blocking calls mirror their blocking counterparts with two differences:

1. **Naming convention:** the letter “I” (for *initiate*) is inserted immediately after the first underscore — e.g., `MPI_Isend`, `MPI_Irecv`.
2. **Request handle:** the final argument is a **request handle** (`MPI_Request`) — an opaque object that tracks the state of the pending operation.

C signatures:

```
int MPI_Isend(const void *buf, int count, MPI_Datatype type,
             int dest, int tag, MPI_Comm comm, MPI_Request *request);

int MPI_Irecv(void *buf, int count, MPI_Datatype type,
             int source, int tag, MPI_Comm comm, MPI_Request *request);
```

The request handle is analogous to a **buzzer at a fast-food restaurant**: you submit your order (initiate communication) and receive a buzzer (request handle); later, you use the buzzer to check whether your order is ready.

0.5.8.2 35.2 Checking for Completion

Three functions operate on request handles:

Function	Behaviour	Analogy
<code>MPI_Wait(request, status)</code>	Blocks until the operation associated with the request completes. The status object is populated upon completion.	Sitting at the table until the buzzer sounds
<code>MPI_Test(request, flag, status)</code>	Non-blocking check: sets flag to true if the operation has completed, false otherwise. Does not block.	Walking to the counter every few minutes to ask
<code>MPI_Request_free(request)</code>	Releases the request without waiting for or checking completion.	Returning the buzzer without picking up the order

`MPI_Wait` is the more commonly used: it provides a clean synchronisation point where the programmer knows the buffer is safe to use.

0.5.8.3 35.3 Advantages and Disadvantages

Advantages	Disadvantages
Enables latency hiding : useful computation can proceed between initiation and completion	The programmer must insert explicit completion checks
Enables communication-computation overlap	Buffer must remain unmodified (send) or unread (receive) until completion is verified
Can resolve deadlock situations that blocking calls cannot	Adds programming complexity

Three reasons to check completion status: 1. You need to **modify or reuse** the send buffer. 2. You want to **release resources** associated with the pending communication. 3. You want to **verify** that the communication succeeded.

0.5.9 36. Deadlock in MPI

0.5.9.1 36.1 Definition and Cause **Deadlock** occurs when two or more processes are each waiting for an event that can only be triggered by one of the other waiting processes, creating a circular dependency from which none can escape. In MPI, deadlock is most common with **blocking communication** when both processes attempt to send before either is ready to receive.

0.5.9.2 36.2 Classic Deadlock Example Consider two processes attempting to exchange data using blocking sends:


```

If (rank == 0) Then
    Call MPI_Send(buffer1, 1, MPI_INTEGER, 1, 10, MPI_COMM_WORLD, error)
    Call MPI_Recv(buffer2, 1, MPI_INTEGER, 1, 20, MPI_COMM_WORLD, status, error)
Else If (rank == 1) Then
    Call MPI_Send(buffer2, 1, MPI_INTEGER, 0, 20, MPI_COMM_WORLD, error)
    Call MPI_Recv(buffer1, 1, MPI_INTEGER, 0, 10, MPI_COMM_WORLD, status, error)
End If

```

Why this deadlocks: - Process 0 calls `MPI_Send` → blocks, waiting for process 1 to post a matching receive. - Process 1 calls `MPI_Send` → blocks, waiting for process 0 to post a matching receive. - Neither process reaches its `MPI_Recv` call → **deadlock**.

Caveat: this code may appear to work for small messages if the MPI implementation uses the eager protocol (buffering small messages internally). However, it will deadlock for large messages or on implementations with stricter synchronous semantics. Code that “works by accident” due to implementation-specific buffering is **non-portable** and should be considered incorrect.

0.5.9.3 36.3 Solutions to Deadlock Four strategies resolve the deadlock scenario:

0.5.9.3.1 Solution A — Buffered Send (MPI_Bsend) Replace `MPI_Send` with `MPI_Bsend`: the send copies data to a user-supplied buffer and returns immediately, allowing the process to proceed to its receive.

```

Call MPI_Bsend(buffer1, 1, MPI_INTEGER, 1, 10, MPI_COMM_WORLD, error)
Call MPI_Recv(buffer2, 1, MPI_INTEGER, 1, 20, MPI_COMM_WORLD, status, error)

```

Trade-off: requires extra buffer memory and an additional copy — **not efficient for large data sets**.

0.5.9.3.2 Solution B — Non-Blocking Send (MPI_Isend) Replace `MPI_Send` with `MPI_Isend`: the send initiates without blocking, allowing the process to proceed to its receive.

```

Call MPI_Isend(buffer1, 1, MPI_INTEGER, 1, 10, MPI_COMM_WORLD, request, error)
Call MPI_Recv(buffer2, 1, MPI_INTEGER, 1, 20, MPI_COMM_WORLD, status, error)
Call MPI_Wait(request, status) ! Ensure send has completed

```

Trade-off: more efficient than buffered send, but requires explicit completion checking via `MPI_Wait`.

0.5.9.3.3 Solution C — Reorder Send and Receive (Recommended) The simplest and most efficient solution: rearrange the order of calls so that one process sends first while the other receives first.

```

If (rank == 0) Then
    Call MPI_Send(buffer1, 1, MPI_INTEGER, 1, 10, MPI_COMM_WORLD, error)
    Call MPI_Recv(buffer2, 1, MPI_INTEGER, 1, 20, MPI_COMM_WORLD, status, error)
Else If (rank == 1) Then
    Call MPI_Recv(buffer1, 1, MPI_INTEGER, 0, 10, MPI_COMM_WORLD, status, error)
    Call MPI_Send(buffer2, 1, MPI_INTEGER, 0, 20, MPI_COMM_WORLD, error)
End If

```

Process 0 sends while process 1 receives → the send/receive pair matches immediately. Then process 1 sends while process 0 receives. **No deadlock, no extra buffers, no additional overhead.** This is the **recommended approach** whenever possible.

0.5.9.3.4 Solution D — Combined SendRecv MPI provides `MPI_Sendrecv`, which combines a blocking send and receive into a **single atomic operation**:

```

int MPI_Sendrecv(
    const void *sendbuf, int sendcount, MPI_Datatype sendtype,
    int dest, int sendtag,
    void *recvbuf, int recvcount, MPI_Datatype recvtype,
    int source, int recvtag,
    MPI_Comm comm, MPI_Status *status);

```

Parameter	Description
<code>sendbuf, sendcount, sendtype</code>	Description of the data to send
<code>dest, sendtag</code>	Destination rank and tag for the outgoing message
<code>recvbuf, recvcount, recvtype</code>	Description of the receive buffer
<code>source, recvtag</code>	Source rank and tag for the incoming message
<code>comm</code>	Communicator
<code>status</code>	Status object for the received message

`MPI_Sendrecv` is particularly useful for **swap** communications (two processes exchanging data) and **chain** communications (messages passed along a sequence of processes, e.g., in a ring topology).

A variant, `MPI_Sendrecv_replace`, uses a **single buffer** for both the outgoing and incoming message, with the system handling the internal buffering needed for the exchange.

0.5.9.4 36.4 General Deadlock Prevention Strategies

Strategy	Principle
Careful ordering (Solution C)	Always the first approach to consider; restructure the communication pattern to eliminate circular dependencies
Buffered send (Solution A)	Decouples send from receive timing via user buffers
Non-blocking calls (Solution B)	Each process posts a non-blocking receive before any send, ensuring messages can always be accepted
Combined SendRecv (Solution D)	Atomically handles bidirectional exchange, letting MPI manage the internal sequencing

Key insight: deadlock is most often the result of **not carefully thinking through the parallelisation scheme**. Solution C — reordering calls — should always be the first strategy considered. The other strategies are fallbacks for situations where reordering is not feasible (e.g., complex communication topologies).

0.5.10 Summary of Key Concepts — Lecture 5

Concept	Key Takeaway
MPI	A <i>de facto</i> standard specification for message passing; implemented as a library (not a language extension)
Implementations	MPICH (reference), OpenMPI (widely used), Intel MPI (Intel-optimised)
MPI_Init / MPI_Init_thread	Must be called before any MPI operation; <code>MPI_Init_thread</code> negotiates thread-safety level
Communicator	Group + context; <code>MPI_COMM_WORLD</code> is the default; custom communicators isolate communication contexts
Rank and Size	Rank $[0, P-1]$ identifies a process; size = total number of processes
MPI Datatypes	Portable type descriptors (e.g., <code>MPI_INT</code> , <code>MPI_DOUBLE</code>); derived types for non-contiguous data
Tags	Integer labels for message filtering at the receiver
MPI_Send / MPI_Recv	Blocking point-to-point operations; send/receive triple: (buffer, count, datatype)
Send modes	Standard (implementation-defined), Synchronous (waits for receiver), Buffered (copies to user buffer), Ready (assumes receiver ready)

Concept	Key Takeaway
Non-blocking (<code>MPI_Isend</code> / <code>MPI_Irecv</code>)	Initiate communication and return immediately; require <code>MPI_Wait</code> or <code>MPI_Test</code> for completion
Message overhead	System overhead (data transfer) + synchronisation overhead (waiting for partner)
Deadlock	Circular wait among blocking calls; resolved by reordering, buffering, non-blocking calls, or <code>MPI_Sendrecv</code>
<code>MPI_Sendrecv</code>	Combined send+receive in one call; eliminates deadlock in exchange patterns
Best practice	Design for <code>MPI_Ssend</code> correctness, deploy with <code>MPI_Send</code> ; always duplicate communicators in libraries

0.6 Lecture 6 — MPI Collective Operations

0.6.1 37. Introduction to Collective Operations

While point-to-point communication (Lecture 5) involves explicit message exchanges between **pairs** of processes, many parallel algorithms require coordinated communication across **entire groups** of processes. **Collective operations** provide a higher-level abstraction for these patterns, replacing what would otherwise be complex sequences of point-to-point calls with single, semantically clear function invocations.

0.6.1.1 37.1 Definition and Characteristics A **collective operation** is a communication routine in which **all processes** within a communicator participate. Every process in the specified communicator must invoke the same collective call; failure to do so results in deadlock or undefined behaviour.

Key characteristics that distinguish collective operations from point-to-point communication:

Property	Detail
Scope	Involves all processes in the specified communicator
Symmetry	Every process executes the same collective call (though roles may differ, e.g., root vs. non-root)
Blocking semantics	For blocking collectives, each process blocks until it has locally completed — i.e., the send/receive buffers can be safely reused. This does not imply global synchronisation (except for <code>MPI_Barrier</code>)

Property	Detail
Synchronisation	May or may not synchronise processes internally — this is implementation-dependent
Root process	Some collectives (e.g., <code>MPI_Bcast</code> , <code>MPI_Reduce</code>) designate a single root process that originates or receives data
Data matching	Senders and receivers must specify exactly matching amounts of data
No tags	Collective operations do not use message tags — the communicator and call ordering provide sufficient context for message matching
Non-blocking variants	MPI-1 and MPI-2 provided only blocking collectives; MPI-3 introduced non-blocking collective operations (e.g., <code>MPI_Ibcast</code> , <code>MPI_Ireduce</code>)

0.6.1.2 37.2 Classification MPI collective operations fall into three classes:

Class	Purpose	Examples
Synchronisation	Force all processes to reach a common point before any may proceed	<code>MPI_Barrier</code>
Data movement	Redistribute data among processes	<code>MPI_Bcast</code> , <code>MPI_Scatter</code> , <code>MPI_Gather</code> , <code>MPI_Allgather</code> , <code>MPI_Alltoall</code>
Global computation	Perform arithmetic or logical operations across distributed data	<code>MPI_Reduce</code> , <code>MPI_Allreduce</code> , <code>MPI_Scan</code>

0.6.2 38. Synchronisation: `MPI_Barrier`

0.6.2.1 38.1 Purpose and Semantics `MPI_Barrier` is the simplest collective operation. It implements a **global synchronisation point**: no process in the communicator may proceed past the barrier until **every** process has reached it.

C signature:

```
int MPI_Barrier(MPI_Comm comm);
```

Fortran signature:

```
CALL MPI_BARRIER(comm, ierr)
```

Parameter	Description
<code>comm</code>	The communicator defining the group of processes to synchronise

0.6.2.2 38.2 Execution Model Consider 5 processes (P_0 through P_4) arriving at a barrier at different times:

1. Each process reaches the `MPI_Barrier` call at its own pace, depending on its workload.
2. Processes that arrive early **block** and wait.
3. Once the **last** process (P_4 in a typical scenario) reaches the barrier, all processes are released simultaneously and may continue execution.

0.6.2.3 38.3 When to Use Barriers Despite its conceptual simplicity, `MPI_Barrier` is **almost never required** in a well-designed parallel program. This is because other collective operations (broadcasts, reductions) and point-to-point communication already enforce the necessary ordering constraints. Barriers are primarily useful for:

- **Performance measurement:** ensuring all processes start a timed section simultaneously, so that the measured time reflects computation rather than startup skew.
- **Debugging:** isolating the source of errors by confirming that all processes reach certain checkpoints.
- **Load balancing diagnostics:** measuring the time spent in the barrier reveals the degree of load imbalance — processes that arrive early are idle until the slowest process catches up.

Best practice: if you find yourself inserting barriers to make a program “work correctly,” the underlying communication pattern likely has a design flaw. Barriers should be used for measurement and debugging, not for correctness.

0.6.3 39. Data Movement Operations

Data movement collectives redistribute data among processes. They replace what would otherwise require multiple point-to-point send/receive pairs, with the MPI implementation free to choose the most efficient internal algorithm (tree-based, pipelined, etc.).

0.6.3.1 39.1 Broadcast: MPI_Bcast **Broadcast** is a **one-to-all** communication pattern: one process (the **root**) sends the same data to every other process in the communicator.

C signature:

```
int MPI_Bcast(void *buf, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm);
```

Fortran signature:

CALL MPI_BCAST(buf, count, type, root, comm, ierr)

Parameter	Description
buf	Address of the data buffer. On the root process, this contains the data to be sent. On all other processes, this is the buffer where the received data will be stored
count	Number of elements in the buffer
datatype	MPI datatype of each element
root	Rank of the process that originates the broadcast
comm	Communicator

Key constraints: - All processes must specify the **same root**, the **same count**, and the **same communicator**. - After the call returns, every process holds an identical copy of the data originally held by the root.

Example (Fortran): Broadcasting a 2-element real array from process 0 to all processes:

```
root = 0
IF (myid .EQ. 0) THEN
    a(1) = 2.0
    a(2) = 4.0
END IF
CALL MPI_BCAST(a, 2, MPI_REAL, 0, MPI_COMM_WORLD, ierr)
```

After this call, every process in MPI_COMM_WORLD has a(1) = 2.0 and a(2) = 4.0, regardless of what values they held previously.

0.6.3.2 39.2 Scatter: MPI_Scatter Scatter is a **one-to-all** communication pattern where the root process partitions its data into equal-sized segments and distributes one segment to each process (including itself).

C signature:

```
int MPI_Scatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
               void *recvbuf, int recvcount, MPI_Datatype recvtype,
               int root, MPI_Comm comm);
```

Fortran signature:

CALL MPI_SCATTER(sndbuf, sndcount, sndtype, rcvbuf, rcvcount, rcvtype, root, comm, ierr)

Parameter	Description
<code>sendbuf</code>	Address of the send buffer (significant only at root)
<code>sendcount</code>	Number of elements sent to each process (not the total buffer size)
<code>sendtype</code>	MPI datatype of the send buffer elements
<code>recvbuf</code>	Address of the receive buffer (all processes)
<code>recvcount</code>	Number of elements in the receive buffer
<code>recvtype</code>	MPI datatype of the receive buffer elements
<code>root</code>	Rank of the sending process
<code>comm</code>	Communicator

Important: `sendcount` specifies the number of elements sent **to each individual process**, not the total number of elements in `sendbuf`. The total size of `sendbuf` at the root must be `sendcount` $\times$ P , where P is the number of processes in the communicator.

The sender arguments (`sendbuf`, `sendcount`, `sendtype`) are significant **only at the root** process; other processes may pass arbitrary values for these parameters (though in practice, the same call signature is used everywhere for clarity).

Example (Fortran): Scattering 16 elements across 8 processes, 2 elements per process:

```

root = 0
IF (myid .EQ. root) THEN
    DO i = 1, 16
        a(i) = REAL(i)
    END DO
END IF
nsnd = 2
CALL MPI_SCATTER(a, nsnd, MPI_REAL, b, nsnd, MPI_REAL, root, MPI_COMM_WORLD, ierr)

```

After this call, process k holds $b(1) = 2k+1$ and $b(2) = 2k+2$ (0-indexed ranks).

0.6.3.3 39.3 Gather: MPI_Gather **Gather** is the **inverse of scatter**: each process sends data to the root, which collects all contributions into a single buffer. It is an **all-to-one** communication pattern.

C signature:

```

int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
              void *recvbuf, int recvcount, MPI_Datatype recvtype,
              int root, MPI_Comm comm);

```

Fortran signature:

```

CALL MPI_GATHER(sndbuf, sndcount, sndtype, rcvbuf, rcvcount, rcvtype,
               root, comm, ierr)

```


Parameter	Description
<code>sendbuf</code>	Address of the send buffer (all processes)
<code>sendcount</code>	Number of elements in the send buffer
<code>recvbuf</code>	Address of the receive buffer (significant only at root)
<code>recvcount</code>	Number of elements received from each process (not the total)
<code>recvtype</code>	MPI datatype of the receive buffer elements
<code>root</code>	Rank of the receiving process
<code>comm</code>	Communicator

Important: `recvcount` specifies the number of elements collected **from each individual process**. The total size of `recvbuf` at the root must be `recvcount × P`.

The receiver arguments (`recvbuf`, `recvcount`, `recvtype`) are significant **only at the root** process.

Example (Fortran): Each of 8 processes contributes 2 elements; root collects them into a 16-element array:

```

root = 0
b(1) = REAL(myid)
b(2) = REAL(myid)
nsnd = 2
CALL MPI_GATHER(b, nsnd, MPI_REAL, a, nsnd, MPI_REAL, root, MPI_COMM_WORLD, ierr)
IF (myid .EQ. root) THEN
    DO i = 1, (nsnd * nproc)
        WRITE(6,*) myid, ': a(i)=', a(i)
    END DO
END IF

```

0.6.3.4 39.4 Practical Application: Parallel Matrix-Vector Multiplication Gather is commonly used to assemble distributed results. A concrete example is **parallel matrix-vector multiplication**:

Problem setup: - Matrix A of size 100×100 , distributed by rows across 4 MPI processes. - Each process holds 25 rows of A (stored in `Apart[25][100]`). - Vector b of length 100 is replicated on all processes. - Each process computes its local portion of the product $c = Ab$, yielding 25 elements stored in `cpart[25]`.

Computation and collection (C):

```

float Apart[25][100], b[100], cpart[25], ctotat[100];
int root = 0;

// Each process computes its 25 rows of the product
for (int i = 0; i < 25; i++) {
    cpart[i] = 0;
    for (int k = 0; k < 100; k++) {

```

```

        cpart[i] = cpart[i] + Apart[i][k] * b[k];
    }
}

// Collect all partial results on root
MPI_Gather(cpart, 25, MPI_FLOAT, ctotat, 25, MPI_FLOAT,
          root, MPI_COMM_WORLD);

```

After `MPI_Gather`, the root process holds the complete product vector `ctotat[100]`.

0.6.3.5 39.5 Variable-Length Variants: `Gatherv` and `Scatterv` The standard `MPI_Gather` and `MPI_Scatter` require that every process sends or receives the **same number** of elements. In many practical scenarios, the data cannot be divided evenly (e.g., when $N \bmod P \neq 0$). `MPI_Gatherv` and `MPI_Scatterv` address this by allowing a **variable count** of data from/to each process.

The key differences from their fixed-count counterparts: - The **count** argument is replaced by an **integer array** `counts[]`, where `counts[i]` specifies the number of elements for process i . - An additional **displacement array** `displs[]` specifies the offset (in number of elements) in the root's buffer where each process's data begins or is placed.

This provides full flexibility in distributing irregularly partitioned data.

0.6.3.6 39.6 Allgather: `MPI_Allgather` `Allgather` is functionally equivalent to a gather followed by a broadcast: every process contributes data, and every process receives the complete concatenated result. Unlike `MPI_Gather`, there is **no root** — all processes hold the full gathered data upon completion.

C signature:

```

int MPI_Allgather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                 void *recvbuf, int recvcount, MPI_Datatype recvtype,
                 MPI_Comm comm);

```

Parameter	Description
<code>sendbuf</code>	Address of the send buffer (each process)
<code>sendcount</code>	Number of elements sent by each process
<code>recvbuf</code>	Address of the receive buffer (all processes)
<code>recvcount</code>	Number of elements received from each process
<code>comm</code>	Communicator

Note that the **root** argument is **absent** — there is no distinguished process.

The j -th block of the receive buffer on every process is reserved for the data sent by process j . All blocks must be the **same size**.

A variable-length variant, `MPI_Allgatherv`, allows different block sizes per process, analogous to `MPI_Gatherv`.

0.6.3.7 39.7 All-to-All: MPI_Alltoall **All-to-all** (also called a **complete exchange** or **total exchange**) is the most general data movement collective: every process sends **distinct data** to every other process, and every process receives distinct data from every other process.

Formally, the j -th block from process i is received by process j and stored in the i -th block of process j 's receive buffer. This can be visualised as a **transpose** of a distributed matrix of data blocks.

C signature:

```
int MPI_Alltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, int recvcount, MPI_Datatype recvtype,
                MPI_Comm comm);
```

Parameter	Description
sendbuf	Address of the send buffer; must contain sendcount × P elements (one block per destination)
sendcount	Number of elements sent to each process
recvbuf	Address of the receive buffer; will contain recvcount × P elements
recvcount	Number of elements received from each process
comm	Communicator

A variable-length variant, **MPI_Alltoallv**, uses count and displacement arrays for both sends and receives:

```
int MPI_Alltoallv(const void *sendbuf, const int *sendcounts,
                  const int *sdispls, MPI_Datatype sendtype,
                  void *recvbuf, const int *recvcounts,
                  const int *rdispls, MPI_Datatype recvtype,
                  MPI_Comm comm);
```

0.6.3.8 39.8 Summary of Data Movement Operations The following table provides a unified view of all data movement collectives:

Operation	Pattern	Root Required	Key Feature
MPI_Bcast	One-to-all	Yes	Same data to all processes
MPI_Scatter	One-to-all	Yes	Different data to each process
MPI_Gather	All-to-one	Yes	Collect different data from each process
MPI_Allgather	All-to-all (symmetric gather)	No	Every process gets the full gathered result

Operation	Pattern	Root Required	Key Feature
MPI_Alltoall	All-to-all (complete exchange)	No	Every process sends distinct data to every other process

Each of these operations has a variable-length variant (suffix *v*) for handling unequal data distributions.

0.6.4 40. Global Computation Operations

Global computation collectives combine data from all processes using a specified operation (e.g., sum, maximum, logical AND). They are essential for computing global quantities from distributed data — a ubiquitous requirement in scientific computing.

0.6.4.1 40.1 Reduce: MPI_Reduce **MPI_Reduce** performs an **element-wise reduction** across all processes, combining corresponding elements of each process’s input buffer using a specified operation. The result is stored **only on the root** process.

C signature:

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op, int root,
               MPI_Comm comm);
```

Fortran signature:

```
CALL MPI_REDUCE(snd_buf, rcv_buf, count, type, op, root, comm, ierr)
```

Parameter	Description
sendbuf	Address of the input buffer (each process contributes its local data)
recvbuf	Address of the output buffer (significant only at root)
count	Number of elements in the input (and output) buffers
datatype	MPI datatype of the buffer elements
op	The reduction operation to apply
root	Rank of the process that receives the result
comm	Communicator

Important: the reduction is performed **element-wise** across the buffers. If `count > 1`, the operation is applied independently to each element index across all processes. For example, with `count = 2` and `MPI_SUM`:

$$\text{recvbuf}[0] = \sum_{i=0}^{P-1} \text{sendbuf}_i[0], \quad \text{recvbuf}[1] = \sum_{i=0}^{P-1} \text{sendbuf}_i[1]$$

Example (Fortran): Computing the sum of 2-element arrays across all processes:

```

root = 0
a(1) = 2.0
a(2) = 4.0
CALL MPI_REDUCE(a, res, 2, MPI_REAL, MPI_SUM, root, MPI_COMM_WORLD, ierr)
IF (myid .EQ. 0) THEN
    WRITE(6,*) myid, ': res(1)=', res(1), 'res(2)=', res(2)
END IF

```

With 4 processes, `res(1) = 4 × 2.0 = 8.0` and `res(2) = 4 × 4.0 = 16.0` on the root process.

0.6.4.2 40.2 All-Reduce: MPI_Allreduce `MPI_Allreduce` performs the same element-wise reduction as `MPI_Reduce`, but the result is distributed to **all processes**, not just the root. It is functionally equivalent to `MPI_Reduce` followed by `MPI_Bcast`, but implementations typically use more efficient algorithms (e.g., recursive doubling, ring-based algorithms).

C signature:

```

int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm);

```

Fortran signature:

```

CALL MPI_ALLREDUCE(snd_buf, rcv_buf, count, type, op, comm, ierr)

```

Note the absence of the `root` parameter — every process receives the result.

`MPI_Allreduce` is one of the most frequently used collectives in scientific computing, as global quantities (e.g., the global residual in an iterative solver, the total energy in a simulation) are typically needed by every process to make local decisions.

0.6.4.3 40.3 Scan (Prefix Reduction): MPI_Scan `MPI_Scan` computes a **prefix reduction** (also called a **partial reduction** or **inclusive scan**) across the processes. Process k receives the reduction of elements from processes $0, 1, \dots, k$.

Formally, if each process i contributes a value a_i , then after `MPI_Scan`: - Process 0 holds: a_0 - Process 1 holds: $a_0 \oplus a_1$ - Process 2 holds: $a_0 \oplus a_1 \oplus a_2$ - Process k holds: $\bigoplus_{i=0}^k a_i$

where $\oplus$ denotes the specified reduction operation.

Like `MPI_Reduce`, the scan operates **element-wise** across arrays. For example, with `count = 2`, two independent scans are performed — one for each element index.

Use cases: prefix sums are fundamental building blocks in parallel algorithms, including parallel sorting, load balancing, and computing global offsets for distributed data structures.

0.6.4.4 40.4 Predefined Reduction Operations MPI provides a comprehensive set of predefined operations that can be used as the `op` argument in any reduction collective:

MPI Operation	Function	Applicable Types
<code>MPI_MAX</code>	Maximum	Integer, floating-point
<code>MPI_MIN</code>	Minimum	Integer, floating-point
<code>MPI_SUM</code>	Sum	Integer, floating-point, complex
<code>MPI_PROD</code>	Product	Integer, floating-point, complex
<code>MPI LAND</code>	Logical AND	Integer (interpreted as logical)
<code>MPI_BAND</code>	Bitwise AND	Integer, byte
<code>MPI_LOR</code>	Logical OR	Integer (interpreted as logical)
<code>MPI BOR</code>	Bitwise OR	Integer, byte
<code>MPI_LXOR</code>	Logical exclusive OR	Integer (interpreted as logical)
<code>MPI_BXOR</code>	Bitwise exclusive OR	Integer, byte
<code>MPI_MAXLOC</code>	Maximum value and its location (rank)	Special paired datatypes
<code>MPI_MINLOC</code>	Minimum value and its location (rank)	Special paired datatypes

In addition to predefined operations, MPI allows users to define **custom reduction operations** via `MPI_Op_create`, which takes a user-supplied function and registers it as an MPI operation. This is useful for domain-specific reductions (e.g., combining custom data structures).

0.6.5 41. Performance Considerations for Collective Operations

0.6.5.1 41.1 Hidden Communication Costs Collective operations abstract away significant amounts of internal communication. A single call to `MPI_Alltoall` on P processes, for instance, involves $O(P^2)$ logical message exchanges. While the MPI implementation optimises these internally, the programmer must remain aware of the **communication volume** implied by each collective.

0.6.5.2 41.2 Implementation-Dependent Performance The performance of collective operations depends critically on the MPI implementation’s choice of internal algorithm, which in turn depends on:

- **Message size:** small-message and large-message regimes often use different algorithms.

- **Number of processes:** tree-based algorithms scale logarithmically with P ; ring-based algorithms amortise bandwidth costs.
- **Network topology:** the mapping between MPI ranks and physical network locations affects latency.

Because these choices are **implementation-dependent**, the performance of the same collective call can vary significantly across MPI implementations and hardware platforms.

0.6.5.3 41.3 Broadcast: Naïve vs. Optimised Algorithms The difference between naïve and optimised implementations is most clearly illustrated by the broadcast operation:

Naïve approach — the root sends the message to each of the $P - 1$ other processes sequentially:

Metric	Value
Data transferred	$(P - 1) \times p$
Number of steps	$P - 1$
Parallelism	None — only the root participates in sending

This approach is **serialised**: the root becomes a bottleneck, and the total time grows linearly with P .

Tree-based approach — each process that has received the message immediately helps propagate it to others:

Metric	Value
Data transferred	$(P - 1) \times p$
Number of steps	$\lceil \log_2 P \rceil$
Parallelism	Doubles each step — exponential propagation

In the tree-based approach: - Step 1: root sends to 1 process $\rightarrow$ 2 processes have the data. - Step 2: both send simultaneously $\rightarrow$ 4 processes have the data. - Step k : 2^k processes have the data.

The total data transferred is the same, but the number of steps is reduced from $O(P)$ to $O(\log P)$, dramatically improving scalability.

0.6.5.4 41.4 Scatter: Naïve vs. Optimised Algorithms A similar analysis applies to scatter:

Naïve approach — the root sends each segment sequentially:

Metric	Value
Data transferred	$(P - 1) \times p$
Number of steps	$P - 1$

Tree-based approach — the root first sends half the data to one process, then both processes split their halves further:

Metric	Value
Data transferred	$\lceil \log_2 P \rceil \times P \times p/2$
Number of steps	$\lceil \log_2 P \rceil$

In the tree-based scatter: - Step 1: root sends half of all data ($P \times p/2$ bytes) to one helper process. - Step 2: both processes send a quarter of the original data to two new processes. - Step $\lceil \log_2 P \rceil$: each process holds exactly its own segment of size p .

The number of steps scales as $O(\log P)$ rather than $O(P)$, but the **total data transferred is larger** ($\log_2 P \times P \times p/2$ vs. $(P - 1) \times p$). This means the tree-based approach scales better in terms of **latency** (fewer steps), but for very large messages, the additional data redundancy during intermediate steps can become a **bandwidth bottleneck**.

0.6.5.5 41.5 Forced Synchronisation Caveat Because some implementations of collective operations internally synchronise processes (e.g., using barriers or rendezvous protocols), calling a collective may introduce **unexpected idle time**. In latency-sensitive applications, it may occasionally be more efficient to implement the equivalent communication pattern using asynchronous point-to-point calls, allowing computation to overlap with communication.

General guidance: use collective operations as the default — they are typically well-optimised and produce cleaner code. Resort to point-to-point alternatives only when profiling demonstrates that a specific collective is a performance bottleneck.

0.6.6 Summary of Key Concepts — Lecture 6

Concept	Key Takeaway
Collective operations	Communication involving all processes in a communicator; replace complex point-to-point sequences
Three classes	Synchronisation (MPI_Barrier), data movement (MPI_Bcast , MPI_Scatter , MPI_Gather , etc.), global computation (MPI_Reduce , MPI_Scan)
MPI_Barrier	Global synchronisation; rarely needed in well-designed programs; useful for benchmarking
MPI_Bcast	One-to-all: same data from root to all processes
MPI_Scatter / MPI_Gather	One-to-all / all-to-one: distribute or collect distinct data segments; sendcount/recvcount is per-process, not total
MPI_Allgather	Gather + broadcast in one call; every process gets the full result; no root

Concept	Key Takeaway
MPI_Alltoall	Complete exchange: every process sends distinct data to every other process
Variable-length variants	<code>MPI_Scatterv</code> , <code>MPI_Gatherv</code> , <code>MPI_Allgatherv</code> , <code>MPI_Alltoallv</code> — handle unequal data distributions via count and displacement arrays
MPI_Reduce	Element-wise reduction across all processes; result on root only
MPI_Allreduce	Like <code>MPI_Reduce</code> , but result distributed to all processes
MPI_Scan	Prefix (inclusive) reduction: process k gets $\bigoplus_{i=0}^k a_i$
Predefined operations	<code>MPI_SUM</code> , <code>MPI_MAX</code> , <code>MPI_MIN</code> , <code>MPI_PROD</code> , bitwise/logical operations, <code>MPI_MAXLOC</code> / <code>MPI_MINLOC</code>
No tags	Collective operations do not use message tags
Naïve vs. tree-based	Naïve: $O(P)$ steps; tree-based: $O(\log P)$ steps — but may transfer more total data
Performance	Implementation-dependent; hidden communication costs can be significant; profiling is essential

0.7 Lecture 7 — Network Fundamentals for Parallel Computing

0.7.1 42. Modelling Network Performance

While Section 13 introduced the role and classification of networks in an HPC cluster, this lecture develops the **quantitative framework** needed to analyse and predict the performance of inter-node communication — the critical bottleneck in most distributed-memory parallel applications.

0.7.1.1 42.1 The Linear Communication Model The simplest and most widely used model for the time required to transfer a single message across a network is the **linear model**:

$$T_{\text{comm}} = \lambda + \frac{M}{b_{\text{network}}}$$

where: - T_{comm} = total transfer time for the message (seconds), - λ = **network latency** — the fixed startup cost to establish the communication channel and initiate the transfer, independent of message size (seconds), - M = message size (bytes), - b_{network} = **asymptotic network bandwidth** — the maximum sustained data transfer rate of the network link (bytes/second).

Interpretation: - For **very small messages** ($M \rightarrow 0$), the transfer time is dominated by the latency λ . The message size is negligible compared to the fixed overhead of setting up the communication. - For **very large messages** ($M \rightarrow \infty$), the transfer time is dominated by the bandwidth term M/b_{network} . The setup cost becomes insignificant relative to the time spent transmitting data. - The **crossover point** occurs when $M^* = \lambda \cdot b_{\text{network}}$ — messages smaller than M^* are latency-bound, while messages larger than M^* are bandwidth-bound.

This model, while simplistic (it ignores protocol overhead, congestion, and contention), provides a remarkably effective first-order approximation for point-to-point communication costs and is the foundation for most parallel performance models.

0.7.1.2 42.2 The Ping-Pong Benchmark The standard experimental method for determining λ and b_{network} is the **ping-pong** benchmark: two processes on separate nodes exchange a single message of N bytes back and forth.

Algorithm:

1. Process 0 records the wall-clock time S .
2. Process 0 sends a message of N bytes to process 1.
3. Process 1 receives the message and immediately sends it back to process 0.
4. Process 0 receives the returned message and records the wall-clock time E .
5. The **round-trip time** is $E - S$; the **one-way transfer time** is $(E - S)/2$.
6. The **throughput** is $2N/(E - S)$ bytes per second.

Pseudocode:

```
myID = get_process_ID()
if (myID == 0) then
    targetID = 1
    S = get_walltime()
    call Send_message(buffer, N, targetID)
    call Receive_message(buffer, N, targetID)
    E = get_walltime()
    throughput = 2 * N / (E - S)           ! bytes/sec
    one_way_time = (E - S) / 2           ! seconds
else
    targetID = 0
    call Receive_message(buffer, N, targetID)
    call Send_message(buffer, N, targetID)
endif
```

By varying N across a wide range of message sizes and plotting T_{comm} versus N , the latency λ can be estimated as the **y-intercept** (the transfer time for a zero-size message), and the bandwidth b_{network} as the **inverse of the slope** in the large-message regime.

0.7.1.3 42.3 Standard Benchmark Suites Production-quality ping-pong benchmarks (and many other communication patterns) are available in two widely used suites:

Suite	Developer	Repository
IMB (Intel MPI Benchmarks)	Intel	github.com/intel/mpi-benchmarks
OSU Micro-Benchmarks	Ohio State University (MVAPICH group)	mvapich.cse.ohio-state.edu/benchmarks

Both suites automate the sweep over message sizes, handle warm-up iterations, compute statistics, and report latency and bandwidth in standardised formats, making them essential tools for characterising any HPC network.

0.7.2 43. Network Topology

0.7.2.1 43.1 What Is Network Topology? **Network topology** describes the geometric arrangement of connections among the nodes and switches in a communication network. The choice of topology profoundly affects the cost, scalability, and performance of an HPC system.

0.7.2.2 43.2 Key Topology Metrics Three fundamental properties characterise any network topology:

Metric	Definition	Significance
Diameter	The maximum shortest-path distance (in hop count or number of links) between any two nodes in the network	Determines the worst-case latency for communication between any pair of nodes
Nodal degree	The number of links connecting to each node (or switch)	Determines the hardware cost per node; lower degree implies simpler, cheaper nodes
Bisection bandwidth (B_b)	The smallest total bandwidth obtained by partitioning the network into two equal halves of $N/2$ nodes and summing the bandwidths of all links crossing the partition	Provides a general measure of the network's data transfer capability under worst-case global communication patterns

A **good topology** aims for: - **Small diameter** (low worst-case latency), - **Small nodal degree** (low hardware cost), - **Large bisection bandwidth** (high global communication throughput).

These goals are inherently in tension — achieving all three simultaneously is impossible, and every topology represents a specific trade-off.

The **normalised bisection bandwidth** B_b/N_{nodes} is a particularly meaningful metric for scalability analysis: it measures the per-node share of the worst-case global bandwidth. A topology where B_b/N_{nodes} remains constant as the system grows is said to have **constant bisection bandwidth** — the most desirable scaling property.

0.7.2.3 43.3 Common HPC Network Topologies

0.7.2.3.1 Bus Topology The simplest interconnect: all nodes share a single communication medium.

Property	Value
Diameter	1 (all nodes directly connected to the bus)
Nodal degree	1
Bisection bandwidth	$B_b = B$ (the bus bandwidth); $B_b/N_{\text{nodes}} \propto 1/N_{\text{nodes}}$

Advantages: - Low latency for small systems. - Easy to implement.

Disadvantages: - Bandwidth is **shared** among all devices — contention increases linearly with node count. - **Not scalable:** normalised bisection bandwidth degrades as $1/N$. - A single defective agent can block the entire bus. - Example: PCI bus within a node — suitable for connecting a few devices but not for inter-node communication.

0.7.2.3.2 Non-Blocking Crossbar Switch A crossbar switch provides a **dedicated connection** between any input port and any output port simultaneously, allowing multiple concurrent communications without interference.

Property	Value
Diameter	1
Nodal degree	1 (each node connects to one switch port)
Bisection bandwidth	$B_b = (N/2) \times B$ (full bisection bandwidth)

Characteristics: - Enables **non-blocking** communication: any permutation of source-destination pairs can communicate simultaneously at full bandwidth. - Switches can be **cascaded** to form hierarchies for larger systems. - **Cost scales quadratically** with port count ($O(N^2)$ crosspoints), making large crossbar switches prohibitively expensive. - Used as building blocks (leaf switches, spine switches) in hierarchical topologies.

0.7.2.3.3 Meshes and Toruses When fat-tree topologies become prohibitively expensive at very large scales, **meshes** and **toruses** offer a cost-effective compromise.

Key characteristics: - Each node functions as a **router**, with direct connections only to its immediate neighbours. - Communication between non-adjacent nodes requires **multi-hop routing** through intermediate nodes. - Available in multiple dimensions: **2D meshes**, **3D toruses** (where

opposite edges wrap around), and **n-dimensional hypercubes**. - Require **intelligent resource management** and **routing algorithms** to minimise congestion and balance traffic.

Toruses (used notably in Fugaku’s 6D mesh/torus interconnect) provide good locality properties for applications with regular communication patterns (e.g., stencil computations in computational fluid dynamics).

0.7.2.3.4 Fat-Tree and Constant Bisection Bandwidth (CBB) Networks The **fat-tree** topology, first proposed by Charles Leiserson (1985), is the most widely used topology in modern HPC clusters.

Ideal fat-tree: - A hierarchical tree structure where the bandwidth of links **doubles at each level** as one moves from leaf switches toward the root. - Provides **full bisection bandwidth**: $N_{\text{nodes}}/2$ end-to-end connections can communicate simultaneously at full link bandwidth. - Theoretically **fully non-blocking**: any permutation of source-destination pairs can communicate at full speed. - Impractical at large scale because the root switch would need to be an $N \times N$ crossbar.

Practical implementation — CBB (Constant Bisection Bandwidth) networks:

In practice, fat-trees are approximated using hierarchies of smaller, commodity switches:

- **Leaf (edge) switches:** connect groups of compute nodes.
- **Spine switches:** interconnect the leaf switches.

In a two-level CBB network with N nodes and per-link bandwidth B :

$$B_b = B \times \frac{N_{\text{nodes}}}{2}$$

$$\frac{B_b}{N_{\text{nodes}}} = \frac{B}{2} = \text{constant}$$

This constant normalised bisection bandwidth ensures that the network’s per-node communication capacity does not degrade as the system scales — a critical property for large parallel applications.

Most modern commodity HPC clusters use this two- or three-level CBB topology.

0.7.2.4 43.4 Oversubscription In practice, budget and physical constraints often prevent building a fully non-blocking fat-tree. **Oversubscription** occurs when the spine switches cannot support $N_{\text{nodes}}/2$ simultaneous full-bandwidth connections:

$$\frac{B_b}{N_{\text{nodes}}} = \frac{B}{2k}$$

where k is the **oversubscription factor** (or **oversubscription ratio**). For example, $k = 3$ means the bisection bandwidth is only one-third of the full non-blocking capacity.

Implications: - Oversubscribed networks are significantly cheaper to build. - **Resource management** (job placement) becomes critical: the scheduler should attempt to place communicating processes on nodes connected to the same leaf switch, minimising the need for inter-switch traffic.

- Applications with **all-to-all** communication patterns suffer disproportionately on oversubscribed networks, while applications with predominantly **local** communication patterns may see little degradation.

0.7.3 44. High-Speed Interconnects in Modern HPC

0.7.3.1 44.1 The “Big Three” Networks The world’s fastest supercomputers are powered by three major high-speed interconnect technologies:

Network	Developer	Key Deployments	Characteristics
Cray Slingshot-11	HPE (Hewlett Packard Enterprise)	El Capitan (#1), Frontier (#2)	Optimised for congestion management and low latency at extreme scale; backbone of exascale systems
NVIDIA Mellanox InfiniBand	NVIDIA	Leonardo (#7), HPC6 (#5), ORFEO	Most widely used network across the TOP500; dominant in GPU-heavy AI clusters; offers RDMA capability
Slingshot (HPE)	HPE	LUMI (#8)	Bridges traditional Ethernet and specialised HPC networking; Ethernet-compatible at the physical layer

All three technologies operate transparently beneath the MPI implementation — MPI libraries (OpenMPI, Intel MPI, MPICH) are configured to exploit the specific features of the underlying network (RDMA, zero-copy transfers, adaptive routing).

0.7.3.2 44.2 InfiniBand: Architecture and Evolution **InfiniBand (IB)** is the dominant high-speed interconnect in HPC. It uses a **serial data transfer** model with configurable **link width** and **link speed**.

0.7.3.2.1 Link Width InfiniBand supports three standard link widths:

Width	Description
1x	One differential pair per transmit/receive direction

Width	Description
4x	Four differential pairs per direction (most common in HPC)
12x	Twelve differential pairs per direction (used for inter-switch links)

0.7.3.2.2 Link Speed Evolution Each generation of InfiniBand increases the per-lane data rate:

Generation	Year	Encoding	Per-Lane Rate	4x Theoretical	4x Actual Rate
SDR (Single Data Rate)	2001	8b/10b	2.5 Gbit/s	10 Gbit/s	~8 Gbit/s
DDR (Double Data Rate)	2005	8b/10b	5 Gbit/s	20 Gbit/s	~16 Gbit/s
QDR (Quad Data Rate)	2007	8b/10b	10 Gbit/s	40 Gbit/s	~32 Gbit/s
FDR (Fourteen Data Rate)	2011	64b/66b	14.06 Gbit/s	56 Gbit/s	~54.5 Gbit/s
EDR (Enhanced Data Rate)	2014	64b/66b	25.78 Gbit/s	103 Gbit/s	~100 Gbit/s
HDR (High Data Rate)	2018	256b/257b	53.13 Gbit/s	212.5 Gbit/s	~200 Gbit/s
NDR (Next Data Rate)	2022	256b/257b	106.25 Gbit/s	425 Gbit/s	~400 Gbit/s
XDR (eXtreme Data Rate)	2024	256b/257b	212.5 Gbit/s	850 Gbit/s	~800 Gbit/s
GDR (Gxx Data Rate)	2025–2028	TBD	Target: 2× XDR	~1.7 Tbit/s	TBD

The **link rate** is the product of link width and link speed. The difference between theoretical and actual rates arises from the encoding overhead (e.g., 8b/10b encoding transmits 10 bits for every 8 bits of payload, incurring a 20% overhead; 64b/66b encoding reduces this to ~3%; 256b/257b encoding to ~0.4%).

The progression from SDR (2001) to XDR (2024) represents an **80-fold increase** in per-link bandwidth over approximately two decades, closely tracking the growth in computational performance.

0.7.3.2.3 InfiniBand vs. Ethernet While Ethernet is ubiquitous in general-purpose networking, InfiniBand offers significant advantages for HPC workloads:

Property	InfiniBand (HDR/NDR)	High-Speed Ethernet (100/400 GbE)
Latency	~1–2 μ s	~5–50 μ s
Bandwidth	200–400 Gbit/s per port	100–400 Gbit/s per port
RDMA	Native (hardware-level)	Requires RoCE (RDMA over Converged Ethernet) — additional complexity
CPU overhead	Minimal (offloaded to HCA)	Higher (more CPU involvement in protocol processing)
Cost	Higher per-port cost	Lower per-port cost
Ecosystem	HPC-specific	Universal; broader ecosystem

InfiniBand’s key advantage is its **native RDMA (Remote Direct Memory Access)** capability: data can be transferred directly between the memory of two nodes without involving either node’s CPU in the data path, dramatically reducing both latency and CPU overhead.

0.7.4 45. ORFEO Network: Detailed Configuration

The ORFEO cluster’s high-speed network illustrates a practical implementation of the concepts discussed above.

0.7.4.1 45.1 InfiniBand Infrastructure ORFEO’s InfiniBand fabric consists of:

Component	Specification
QM9700 switch	40 ports $\times$ 400 Gbit/s (NDR); aggregate throughput: 51.2 Tb/s
QM8700 switch	40 ports $\times$ 200 Gbit/s (HDR); aggregate throughput: 16 Tb/s; sub-130 ns switch latency
ConnectX-6 HCA	100 Gbit/s per port (HDR100)
ConnectX-7 HCA	200 Gbit/s per port (HDR/NDR)
Splitter cables	Optical and copper cables that split higher-speed ports into multiple lower-speed connections (e.g., $1 \times 400 \rightarrow 4 \times 100$, $1 \times 400 \rightarrow 2 \times 200$, $1 \times 200 \rightarrow 2 \times 100$)

The two switches are interconnected via 400 Gbit/s links, forming the spine of the network fabric. Splitter cables are used extensively to match the port speeds of the switches to the HCA speeds of individual nodes.

0.7.4.2 45.2 Ethernet Networks

Network	Technology	Purpose
In-band management	25 Gbit Ethernet (10× Dell S5148F switches)	I/O, NFS, job scheduling, software distribution
Out-of-band management	1 Gbit Ethernet	IPMI/BMC remote hardware control

0.7.4.3 45.3 Measured Network Parameters Using the ping-pong benchmark (IMB) on ORFEO’s InfiniBand fabric, the following parameters have been experimentally determined:

$$\lambda = 1.35 \mu\text{s}$$

$$b_{\text{network}} = 12 \text{ GB/s}$$

These values allow users to predict communication costs using the linear model:

$$T_{\text{comm}} = 1.35 \times 10^{-6} + \frac{M}{12 \times 10^9} \quad (\text{seconds})$$

For example, transferring a 1 MB message between two nodes on ORFEO:

$$T_{\text{comm}} = 1.35 \times 10^{-6} + \frac{10^6}{12 \times 10^9} \approx 1.35 \times 10^{-6} + 83.3 \times 10^{-6} \approx 84.7 \mu\text{s}$$

For a 1 KB message:

$$T_{\text{comm}} = 1.35 \times 10^{-6} + \frac{10^3}{12 \times 10^9} \approx 1.35 \times 10^{-6} + 0.083 \times 10^{-6} \approx 1.43 \mu\text{s}$$

The first example is clearly **bandwidth-dominated**, while the second is **latency-dominated** — illustrating the two regimes of the linear model.

The network can be assumed to be **full non-blocking** (CBB topology), meaning that $P/2$ pairs of nodes can communicate in parallel at full speed simultaneously.

0.7.4.4 45.4 Interpreting Ping-Pong Results The experimental ping-pong results on ORFEO exhibit a characteristic two-phase pattern:

1. **Small-message regime** (up to a few kilobytes): transfer time is nearly constant at approximately $\lambda \approx 1.35 \mu\text{s}$, dominated by latency. Throughput increases linearly with message size in this regime because more data is transferred for the same fixed startup cost.
2. **Large-message regime** (beyond tens of kilobytes): throughput saturates at the asymptotic bandwidth $b_{\text{network}} \approx 12 \text{ GB/s}$. Transfer time increases linearly with message size, and the latency contribution becomes negligible.

The transition between these regimes typically occurs around $M^* = \lambda \times b_{\text{network}} \approx 1.35 \times 10^{-6} \times 12 \times 10^9 \approx 16$ KB — messages below this size are latency-bound, and messages above it are bandwidth-bound.

0.7.5 46. Implications for MPI Application Performance

0.7.5.1 46.1 Communication Cost in Parallel Algorithms Combining the network performance model with the parallel performance framework from Lecture 4, the total execution time of a parallel application can be decomposed as:

$$T_P = T_{\text{comp}} + T_{\text{comm}} + T_{\text{idle}}$$

where: - T_{comp} = computation time (decreases with P for parallelisable work), - T_{comm} = communication time (typically increases with P as more inter-node messages are required), - T_{idle} = idle time due to load imbalance and synchronisation.

For a program that performs K point-to-point communications of average size M :

$$T_{\text{comm}} \approx K \left(\lambda + \frac{M}{b_{\text{network}}} \right)$$

This expression reveals two optimisation strategies:

1. **Reduce the number of messages (K):** aggregate small messages into fewer, larger ones to amortise the latency cost. This is particularly effective in the latency-bound regime.
2. **Reduce the message size (M):** communicate only the data that is strictly necessary, minimising bandwidth consumption. This is critical in the bandwidth-bound regime.

These two strategies are often in tension — aggregating messages increases M while decreasing K , and the optimal balance depends on the specific values of λ and b_{network} for the target system.

0.7.5.2 46.2 The Communication-to-Computation Ratio A key metric for predicting the scalability of a parallel application is the **communication-to-computation ratio**:

$$\rho = \frac{T_{\text{comm}}}{T_{\text{comp}}}$$

- If $\rho \ll 1$: the application is **compute-bound** and will scale well.
- If $\rho \gg 1$: the application is **communication-bound** and will exhibit poor scaling.
- The ratio ρ typically **increases** with P (as more communication is needed and less work is assigned per processor), eventually dominating the execution time and causing the speed-up curve to flatten or decline — consistent with the communication-augmented Amdahl's Law discussed in Section 24.3.

0.7.6 Summary of Key Concepts — Lecture 7

Concept	Key Takeaway
Linear communication model	$T_{\text{comm}} = \lambda + M/b_{\text{network}}$; latency-dominated for small messages, bandwidth-dominated for large ones
Ping-pong benchmark	Standard method for measuring λ and b_{network} ; available in IMB and OSU benchmark suites
Network topology	Characterised by diameter, nodal degree, and bisection bandwidth
Bus	Simple, low-cost, but shared bandwidth ($B_b/N \propto 1/N$); not scalable
Crossbar switch	Non-blocking, full bisection bandwidth, but $O(N^2)$ cost
Fat-tree / CBB	Hierarchical switch topology; constant normalised bisection bandwidth; dominant in commodity HPC clusters
Oversubscription	Reduces cost by under-provisioning spine bandwidth; factor k reduces B_b/N by $1/k$; makes job placement critical
InfiniBand	Dominant HPC interconnect; SDR (2001) through XDR (2024); native RDMA; $\sim 80\times$ bandwidth increase over two decades
InfiniBand vs. Ethernet	IB: lower latency ($\sim 1 \mu\text{s}$ vs. $\sim 10\text{--}50 \mu\text{s}$), native RDMA, lower CPU overhead; Ethernet: lower cost, broader ecosystem
ORFEO network	HDR + NDR InfiniBand; QM8700 + QM9700 switches; measured $\lambda = 1.35 \mu\text{s}$, $b = 12 \text{ GB/s}$; full non-blocking
Communication-to-computation ratio	$\rho = T_{\text{comm}}/T_{\text{comp}}$; determines whether an application is compute-bound or communication-bound